

承诺书

我们授权全国大学生数学建模竞赛组委会,可将我们的论文以任何形式进行公开展示(包括进行网上公示,在书籍、期刊和其他媒体进行正式或非正式发表等)。

(指导教师签名意味着对参赛队的行为和论文的真实性负责)

日期: 2024 年 9 月 5 日

(请勿改动此页内容和格式。此承诺书打印签名后作为纸质论文的封面，注意电子版论文中不得出现此页。以上内容请仔细核对，如填写错误，论文可能被取消评奖资格。)

赛区评阅编号：
(由赛区填写)

全国评阅编号：
(全国组委会填写)

2024 高教社杯全国大学生数学建模竞赛

编 号 专 用 页

赛区评阅记录（可供赛区评阅时使用）：

评阅人						
备注						

送全国评阅统一编号：
(赛区组委会填写)

(请勿改动此页内容和格式。此编号专用页仅供赛区和全国评阅使用，参赛队打印后装订到纸质论文的第二页上。注意电子版论文中不得出现此页。)

板凳龙闹元宵

基于等距螺线与分段几何约束的运动建模

摘要

“板凳龙”由多节板凳通过把手首尾相连，运动过程中既受到螺线轨迹约束，又受到相邻把手距离、板凳实体宽度和调头空间边界的共同限制。本文围绕盘入、调头和盘出过程，建立以等距螺线方程、极坐标弧长积分、逐节几何递推、有限宽度碰撞判定和速度比例缩放为核心的统一模型，分别求解各时刻位置速度、无碰撞终止时刻、最小螺距、调头路径以及最大安全头速。

针对问题一，首先用等距螺线方程刻画半径随极角线性变化的盘入轨迹，再由极坐标参数曲线的弧长微元积分得到螺线弧长函数，据此反解龙头前把手的角参数；随后利用相邻把手距离约束逐节递推全队位置，并用差分法计算速度。模型生成了 0–300 s 内全部 224 个把手的位置和速度，位置表规模为 449×302 ，速度表规模为 225×302 ，相邻把手距离最大误差小于 $1.0e-11$ m。

针对问题二，将每节板凳视为有限宽度矩形，依据把手连线确定矩形轴线和法向，并采用分离轴定理判断非相邻板凳是否相交。再定义碰撞判定函数，通过扫描确定临界区间，并用二分法逼近首次碰撞时刻。无碰撞终止时刻为 412.474 s，首次碰撞板对为 [0,8]，终止状态下全队速度约在 0.973–1.000 m/s 之间。

针对问题三，在问题二碰撞判定思想的基础上，以龙头附近外侧角点到前方板凳轴线的最小距离作为接触判据。采用变步长搜索：以 0.55 m 为初始螺距逐步缩小，并在出现接触风险后减小步长继续逼近。满足盘入调头空间且不发生接触风险的最小螺距为 0.45 m。

针对问题四，在螺距 1.7 m、调头空间半径 4.5 m 的条件下，以盘入螺线与调头圆的交点作为调头入点，并令盘出螺线与盘入螺线关于中心对称。根据两圆弧相切模型约束两段圆弧分别与盘入、盘出螺线相切、两圆弧相切以及半径比为 2:1 的几何关系，从而确定调头路径。两段圆弧半径分别为 3.005 m 和 1.503 m，调头路径长度为 13.621 m；同时输出调头前后 100 s 内每秒全队位置和速度。

针对问题五，在问题四固定路径上计算单位龙头速度下的全队速度矩阵。由于路径固定时各把手速度随龙头恒速成比例缩放，采用单位头速归一化得到最大速度放大倍数，并反推出龙头速度上界。单位头速下最大速度放大倍数为 1.422，限制把手为第 1 节龙身，对应龙头最大恒定速度为 1.406 m/s。

关键词：等距螺线 几何递推 碰撞判定 二分搜索

一、问题重述

题目给出一条由 223 节板凳组成的“板凳龙”，要求在盘入、调头和盘出三个阶段中，结合把手约束、板凳宽度与调头空间边界，完成位置、速度、终止时刻、最小螺距、调头曲线和最大安全速度等计算。

从建模角度看，问题的核心不在于单个局部几何量，而在于如何把“龙头沿路径运动”与“整队板凳的相邻把手约束”统一到同一套参数化框架中。为此，本文先以等距螺线刻画盘入路径，再用逐节递推恢复全队状态；随后把每节板凳视作有限宽度矩形，利用碰撞检测判定运动可行性；最后围绕调头曲线长度与速度上限，分别转化为两圆弧几何构造和速度缩放问题。

问题一：在螺距为 0.55 m、龙头前把手速度恒为 1 m/s 的条件下，给出 0–300 s 内全部把手的位置和速度。

问题二：沿问题一的盘入螺线继续运动，求首次发生碰撞前的终止时刻，并输出终止状态。

问题三：在半径为 4.5 m 的调头空间内，求使龙头前把手能够盘入边界的最小螺距。

问题四：在螺距为 1.7 m 的条件下，以螺线与调头圆的交点为调头入点，构造调头空间内的 S 形连接曲线。

问题五：在问题四路径上，求满足所有把手速度均不超过 2 m/s 的龙头最大恒定速度。

二、问题分析

五个问题共享同一条主线：先确定龙头轨迹，再由把手间的几何约束回推整队状态，最后用碰撞与速度条件筛选可行解。

对问题一和问题二，关键在于建立“弧长–参数–坐标”的转换。龙头沿等距螺线运动时，弧长是时间的线性函数；只要能解出当前时刻对应的螺线参数，就能顺次恢复每节板凳的把手位置。问题二在此基础上增加了板凳实体宽度，因此不能只看把手中心线，而必须将每节板凳近似为有厚度的矩形，并检测非相邻板凳之间是否发生接触。

对问题三，最小螺距问题本质上是一个单变量可行性搜索。螺距越小，螺线越紧，整队越容易在调头空间边界附近产生干涉；因此可以把“给定螺距是否可行”作为判定函数，再用二分搜索逼近最小可行值。

对问题四，调头曲线由两段相切圆弧组成，且半径满足比例约束。入点由螺线与调头圆交点确定后，切线方向、圆弧半径和圆心角均可由几何关系推出。

对问题五，路径几何固定后，头部恒速会线性传递到全队各把手的速度幅值。于是只需先计算单位头速下的速度放大倍数，再通过“最大值不超过 2 m/s”反推出头速上界即可。

三、 总体分析

本题的关键在于把连续运动、几何约束和路径参数放进同一套语言中统一处理。Q1 与 Q2 共用盘入螺线和把手递推，差别只是 Q2 进一步引入了板凳实体宽度；Q3 则把螺距判定改写为角点到板凳轴线距离的临界搜索；Q4 把调头部分抽象为固定交点下的两段相切圆弧；Q5 在固定 Q4 路径后，通过速度缩放反推可接受的头速上限。

因此，全文的建模顺序应当是“先路径、后约束、再搜索”。先通过螺线弧长和递推关系确定运动状态，再通过几何判定筛选可行区域，最后对螺距或速度上界进行数值求解。这个顺序也与题目给出的五个问题自然对应。

四、 模型假设

为使模型既可计算又不过度偏离题意，本文作如下假设。

1. 板凳在运动中视为刚体，单节板凳的几何尺寸在整个过程中保持不变。
2. 每节板凳的两个把手中心位置固定，且相邻节之间仅通过把手连接。
3. 盘入和盘出阶段的路径均可视为理想平面曲线，忽略弹性、摩擦和摆动等细节。
4. 除题目允许的相邻连接外，任意两节非相邻板凳一旦实体相交即视为发生碰撞。
5. 数值计算中的极小残差仅来自离散化与求根误差，不改变几何判定的结论。

上述假设中，第 4 条是碰撞判定的核心，第 5 条用于说明数值结果的适用边界。它们共同保证后续结论具有明确的几何意义。

五、符号说明

符号	含义
N	板凳总节数, $N = 223$
t	时间, 单位为 s
p	螺距, 单位为 m
v_H	龙头前把手速度, 单位为 m/s
$P_i(t)$	第 i 个把手在时刻 t 的平面坐标
ℓ_H	龙头板前后把手中心距, 2.86 m
ℓ_B	普通板和龙尾板前后把手中心距, 1.65 m
$r(\theta)$	等距螺线的极坐标半径函数
$S(\theta)$	螺线从起点到参数 θ 的弧长
R_T	调头空间半径, 4.5 m
T_{stop}	问题二的无碰撞终止时刻
L_{turn}	调头曲线长度
v_{max}	问题五的龙头最大恒定速度

其中, $P_i(t)$ 既可表示把手中心, 也可在需要时表示与其绑定的板凳参考点, 具体含义在各问题中保持一致。本文所有长度单位均为 m, 速度单位均为 m/s, 角度单位均为 rad。

六、问题一：盘入运动状态模型的建立与求解

6.1 问题一的建模思路

问题一要求给出 0–300 s 内龙头、龙身和龙尾各把手的位置与速度。由于所有把手均约束在同一条盘入螺线上, 而相邻板凳之间又由固定把手间距连接, 因此可将问题拆为两个层次: 先确定龙头前把手的螺线参数, 再沿同一条曲线递推其余把手的参数。

本文采用极坐标描述螺线。原因在于, 题目给出的轨迹本质上是等距螺线, 其核心几何量是“半径随极角线性变化”; 这一关系在极坐标下直接写成代数式, 而在直角坐标中则会引入额外的旋转与曲率表达。故位置求解的关键量由二维坐标降为单一参数 θ 。

6.2 等距螺线方程的建立

设螺线中心为原点 O ， x 轴水平向右， y 轴竖直向上。根据题图约定，点 A 位于正 x 轴上，龙头沿顺时针方向向内盘入。等距螺线满足

$$r(\theta + 2\pi) - r(\theta) = p.$$

若取半径关于极角线性变化，则有

$$r(\theta) = b\theta, \quad b = \frac{p}{2\pi}.$$

问题一中 $p = 0.55 \text{ m}$ ，所以

$$b = \frac{0.55}{2\pi}.$$

将极坐标转为直角坐标，得到把手中心的位置表达式

$$\begin{cases} x(\theta) = r(\theta) \cos \theta, \\ y(\theta) = r(\theta) \sin \theta. \end{cases}$$

因此，一旦确定某一把手对应的螺线参数 θ ，其平面位置就被唯一确定。

为直观说明上述极坐标建模过程，等距螺线的几何要素示意图如下。

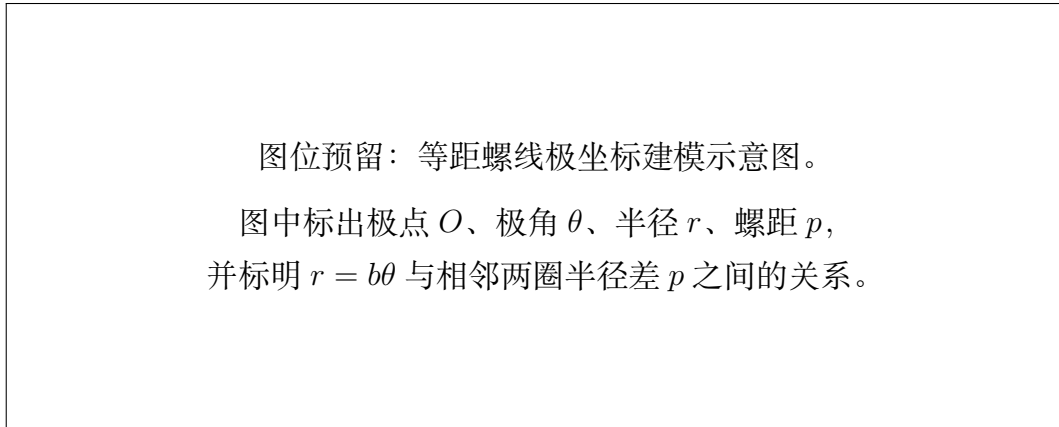


图 1 等距螺线极坐标建模示意

6.3 龙头前把手的弧长关系

题目给定龙头前把手的速度为 1 m/s 。对于参数曲线

$$x(\theta) = b\theta \cos \theta, \quad y(\theta) = b\theta \sin \theta,$$

其导数为

$$\frac{dx}{d\theta} = b \cos \theta - b\theta \sin \theta, \quad \frac{dy}{d\theta} = b \sin \theta + b\theta \cos \theta.$$

故弧长微元满足

$$ds = \sqrt{\left(\frac{dx}{d\theta}\right)^2 + \left(\frac{dy}{d\theta}\right)^2} d\theta = b\sqrt{1 + \theta^2} d\theta.$$

于是从 0 到 θ 的弧长函数为

$$S(\theta) = \int_0^\theta b\sqrt{1 + u^2} du = \frac{b}{2} \left[\theta\sqrt{\theta^2 + 1} + \operatorname{asinh}(\theta) \right].$$

该函数把螺线参数与沿曲线行进距离联系起来，因此龙头位置可由弧长反解得到。

6.4 初始角 θ_0 的确定

题目说明龙头从第 16 圈处开始盘入。等距螺线绕原点一圈对应极角变化 2π ，第 16 圈对应

$$\theta_0 = 16 \times 2\pi = 32\pi.$$

在本文的坐标约定下， $\theta_0 = 32\pi$ 对应正 x 轴方向，与题图中 A 点位于正 x 轴一致。由于龙头顺时针向内运动，半径随时间减小，等价于 θ 随时间减小。

设龙头前把手在时刻 t 的参数为 $\theta_H(t)$ ，则

$$S(\theta_0) - S(\theta_H(t)) = v_H t, \quad v_H = 1.$$

即

$$S(\theta_H(t)) = S(\theta_0) - v_H t.$$

由此得到一元单调方程。求得 $\theta_H(t)$ 后，再代入坐标转换式即可恢复龙头前把手的位置。

6.5 相邻把手递推模型

龙头前把手确定后，其余把手必须满足相邻把手中心距离固定。龙头板的前后把手中心距为

$$\ell_H = 3.41 - 2 \times 0.275 = 2.86 \text{ m},$$

普通龙身板和龙尾板的前后把手中心距为

$$\ell_B = 2.20 - 2 \times 0.275 = 1.65 \text{ m}.$$

相邻把手、板凳实体与中心线之间的几何关系示意图如下，该图用于说明后续递推约束与碰撞矩形模型的共同几何来源。

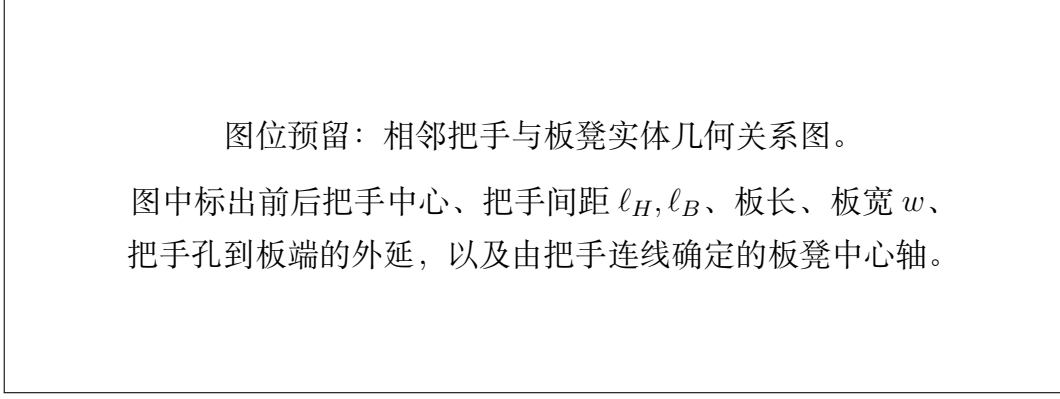


图2 相邻把手与板凳实体的几何关系示意

记第 i 个把手在时刻 t 对应的螺线参数为 $\theta_i(t)$ ，位置为

$$P_i(t) = (x(\theta_i(t)), y(\theta_i(t))).$$

已知 $P_i(t)$ 后，第 $i+1$ 个把手仍位于同一条螺线上，且满足

$$\|P_{i+1}(t) - P_i(t)\|_2 = d_i, \quad d_i = \begin{cases} \ell_H, & i = 0, \\ \ell_B, & i = 1, 2, \dots, 222. \end{cases}$$

展开得

$$[x(\theta_{i+1}) - x(\theta_i)]^2 + [y(\theta_{i+1}) - y(\theta_i)]^2 = d_i^2.$$

由于板凳龙由头至尾单向排布，后一把手在螺线上对应更外侧的参数值，即 $\theta_{i+1} > \theta_i$ 。在该单调区间内求解上式，可排除非物理解。于是，问题一的几何状态由“路径约束 + 间距约束”唯一确定。

6.6 速度计算

对后续把手，速度按相邻时刻的轨迹长度差分得到。若记第 i 个把手在相邻时刻的位置为 $P_i(t - \Delta t)$ 和 $P_i(t + \Delta t)$ ，则可用中心差分近似

$$v_i(t) \approx \frac{\|P_i(t + \Delta t) - P_i(t - \Delta t)\|_2}{2\Delta t}.$$

端点时刻使用单侧差分。该做法与逐秒输出要求一致，也避免了显式求解隐函数导数。

6.7 求解流程

Algorithm 1 问题一逐时刻位置与速度求解流程

Input: 螺距 $p = 0.55$, 整数时刻 $t = 0, 1, \dots, 300$, 把手间距 d_i 。

Output: 全部把手的位置表与速度表。

- 1: 令 $b = p/(2\pi)$, 建立等距螺线 $r(\theta) = b\theta$ 。
 - 2: 对每个时刻 t , 求解弧长方程 $S(\theta_0) - S(\theta_H(t)) = v_H t$, 得到龙头前把手参数 $\theta_H(t)$ 。
 - 3: 对 $i = 1, \dots, 223$, 在 $\theta_i > \theta_{i-1}$ 分支上求解 $\|P_i(t) - P_{i-1}(t)\|_2 = d_i$ 。
 - 4: 将所有 $\theta_i(t)$ 代入坐标变换, 得到 $(x_i(t), y_i(t))$ 。
 - 5: 对离散位置序列作差分, 得到各把手速度 $v_i(t)$ 。
 - 6: **return** 位置表与速度表。
-

上述流程把“轨迹反解”和“链式递推”分离开来, 便于逐时刻稳定求解。
根据该流程得到的板凳龙在盘入螺线上的某一时刻整体排布示意图如下。

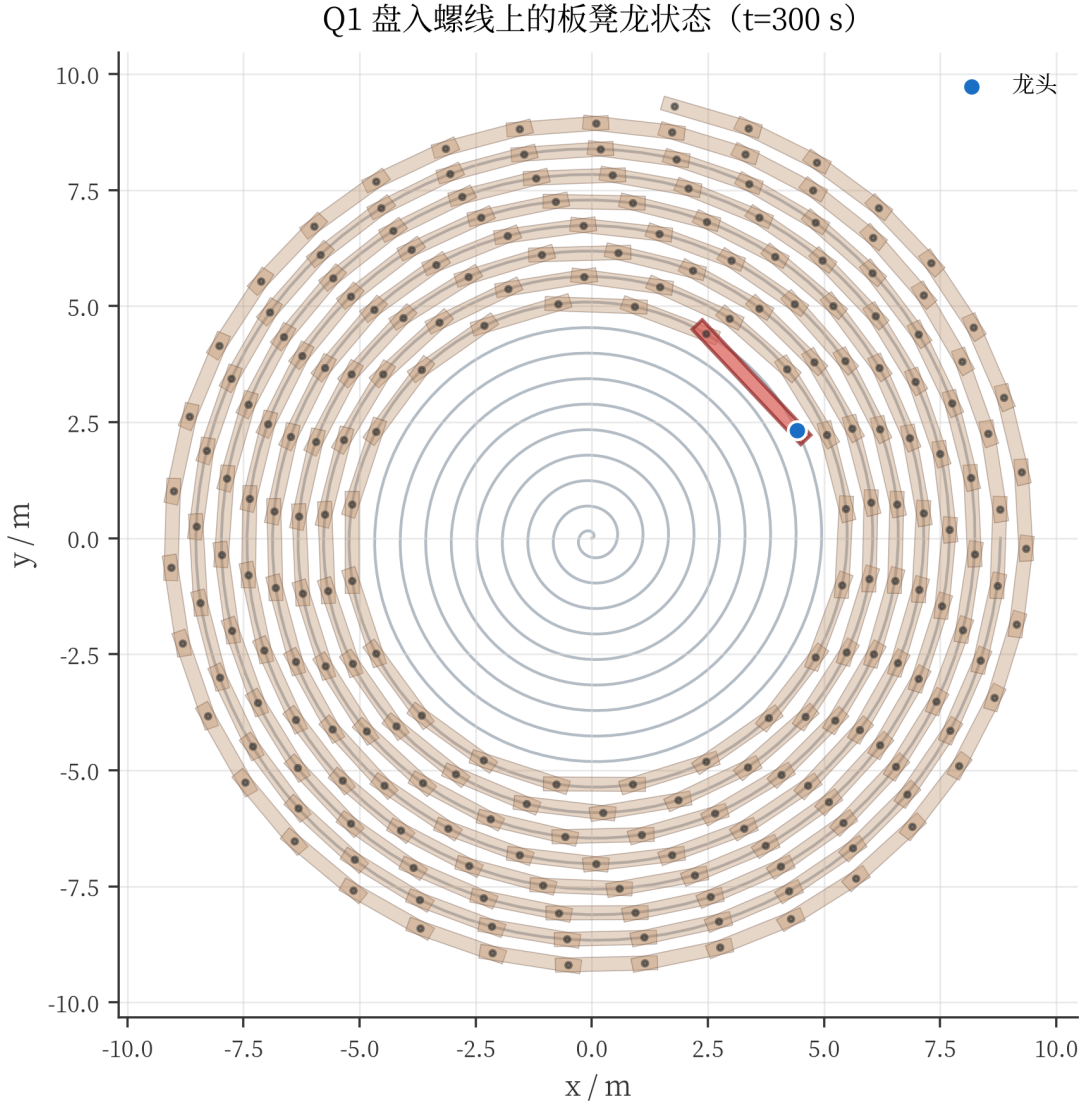


图 3 板凳龙在盘入螺线上的运动状态截取

6.8 结果分析

按上述模型输出的 0–300 s 数据中，位置表规模为 449 行、302 列，速度表规模为 225 行、302 列，覆盖全部 224 个把手和 301 个整数时刻。相邻把手距离的最大误差为

$$8.32 \times 10^{-12} \text{ m},$$

平均绝对误差为

$$4.64 \times 10^{-14} \text{ m}.$$

这说明逐节递推在数值上很好地保持了板凳连接约束。与稠密折线基线在 0 s、60 s、120 s、180 s、240 s 和 300 s 处比较，最大坐标差为 0.0153 m，说明模型输出与离散路径近似结果保持一致。

表 1 问题一位置抽样表

项目	0 s	60 s	120 s	180 s	240 s	300 s
龙头 x (m)	8.800000	5.799209	-4.084887	-2.963609	2.594494	4.420274
龙头 y (m)	0.000000	-5.771092	-6.304479	6.094780	-5.356743	2.320429
第 1 节龙身 x (m)	8.363824	7.456758	-1.445473	-5.237118	4.821221	2.459489
第 1 节龙身 y (m)	2.826544	-3.440399	-7.405883	4.359627	-3.561949	4.402476
第 51 节龙身 x (m)	-9.518732	-8.686317	-5.543150	2.890455	5.980011	-6.301346
第 51 节龙身 y (m)	1.341137	2.540108	6.377946	7.249289	-3.827758	0.465829
第 101 节龙身 x (m)	2.913983	5.687116	5.361939	1.898794	-4.917371	-6.237722
第 101 节龙身 y (m)	-9.918311	-8.001384	-7.557638	-8.471614	-6.379874	3.936008
第 151 节龙身 x (m)	10.861726	6.682311	2.388757	1.005154	2.965378	7.040740
第 151 节龙身 y (m)	1.828753	8.134544	9.727411	9.424751	8.399721	4.393013
第 201 节龙身 x (m)	4.555102	-6.619664	-10.627211	-9.287720	-7.457151	-7.458662
第 201 节龙身 y (m)	10.725118	9.025570	1.359847	-4.246673	-6.180726	-5.263384
龙尾 (后) x (m)	-5.305444	7.364557	10.974348	7.383896	3.241051	1.785033
龙尾 (后) y (m)	-10.676584	-8.797992	0.843473	7.492370	9.469336	9.301164

表 2 问题一速度抽样表

项目	0 s	60 s	120 s	180 s	240 s	300 s
龙头 (m/s)	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000
第 1 节龙身 (m/s)	0.999971	0.999961	0.999945	0.999917	0.999859	0.999709
第 51 节龙身 (m/s)	0.999742	0.999662	0.999538	0.999331	0.998941	0.998065
第 101 节龙身 (m/s)	0.999575	0.999453	0.999269	0.998971	0.998435	0.997302
第 151 节龙身 (m/s)	0.999448	0.999299	0.999078	0.998727	0.998115	0.996861
第 201 节龙身 (m/s)	0.999348	0.999180	0.998935	0.998551	0.997894	0.996574
龙尾 (后) (m/s)	0.999311	0.999136	0.998883	0.998489	0.997816	0.996478

由表2可见，龙头速度保持为 1 m/s，其余关键把手速度随时间略有下降。为更直观比较不同位置把手的速度变化，关键把手速度随时间变化曲线如下。

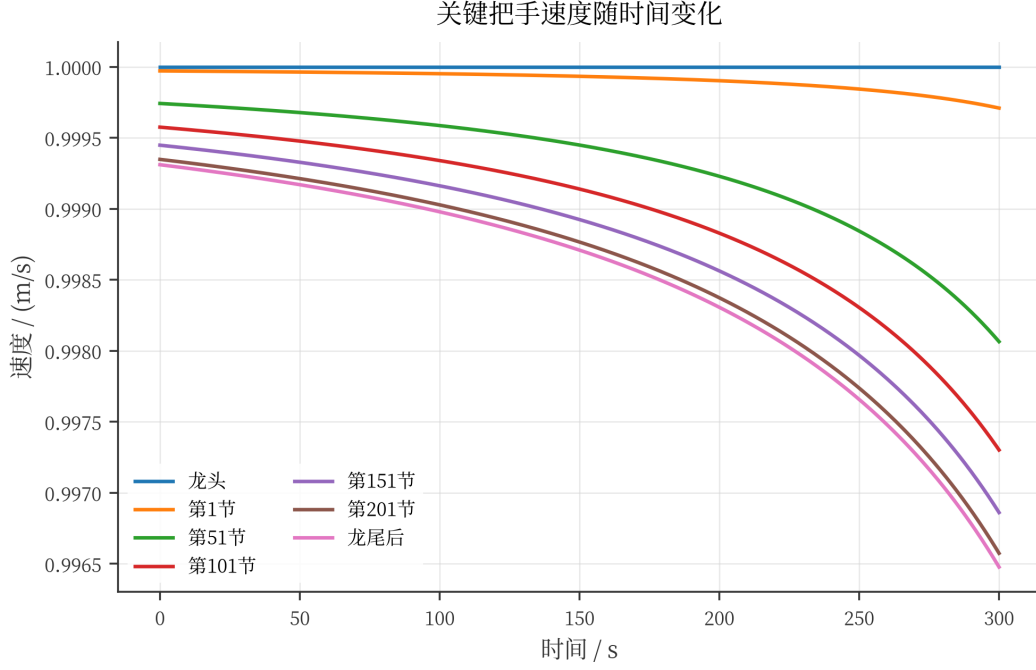


图 4 关键把手速度随时间变化

因此，问题一的等距螺线弧长模型既满足题目对位置和速度输出的要求，也为后续碰撞判定、最小螺距搜索和调头路径建模提供了统一的几何基础。

七、问题二：无碰撞终止时刻的判定

7.1 问题二的建模动机

问题二要求在问题一的盘入轨迹上确定无碰撞终止时刻。若仍只看把手中心线，就只能得到点与点的相对位置，而无法判断实体板凳是否接触；因此必须把每节板凳视为具有长度和宽度的平面刚体。由此，问题二实质上转化为一个几何可行域的临界判定问题。

题目给出的板长、板宽和把手孔中心距板端的偏移量，足以把每节板凳近似成一个有限宽度矩形。该矩形的轴线由前后把手中心连线确定，宽度取 0.30 m。于是“是否碰撞”就可以被写成两个矩形是否有交集的问题。

7.2 矩形板凳模型

对第 i 节板凳，设前后把手中心分别为 P_i^f 和 P_i^b ，则板凳中心轴方向向量可写为

$$\mathbf{u}_i = \frac{P_i^f - P_i^b}{\|P_i^f - P_i^b\|_2},$$

对应的法向单位向量记为 $\mathbf{v}_i$ 。于是，第 i 节板凳可表示为

$$\mathcal{R}_i = \left\{ P_i^c + \xi \mathbf{u}_i + \eta \mathbf{v}_i : |\xi| \leq \frac{L_i}{2}, |\eta| \leq \frac{w}{2} \right\},$$

其中 P_i^c 为板凳中心点， L_i 为该节板凳沿轴线方向的有效长度， $w = 0.30 \text{ m}$ 为板宽。把手中心到板端的偏移固定为 0.275 m ，因此有效长度可由把手间距与两端外延共同确定。

7.3 碰撞约束的数学表达

问题二只检查非相邻板凳。若两节板凳相邻，则它们本就通过把手连接，不应视作碰撞；若两节板凳不相邻，则要求其矩形区域互不相交。故判定条件可写为

$$\mathcal{R}_i \cap \mathcal{R}_j = \emptyset, \quad |i - j| > 1.$$

分离轴定理指出：两个凸多边形不相交，当且仅当存在一条直线，使得两个多边形在该直线法向上的投影区间互不重叠。由于矩形是凸多边形，且其可能的分离轴只需取自两矩形的边法向方向，因此本题中每一对矩形板凳只需检查四个单位方向

$$\mathcal{E}_{ij} = \{\mathbf{u}_i, \mathbf{v}_i, \mathbf{u}_j, \mathbf{v}_j\}.$$

对任意单位方向 $\mathbf{e} \in \mathcal{E}_{ij}$ ，第 i 节板凳在该方向上的投影区间可写为

$$I_i(\mathbf{e}) = [P_i^c \cdot \mathbf{e} - \rho_i(\mathbf{e}), P_i^c \cdot \mathbf{e} + \rho_i(\mathbf{e})],$$

其中投影半径为

$$\rho_i(\mathbf{e}) = \frac{L_i}{2} |\mathbf{u}_i \cdot \mathbf{e}| + \frac{w}{2} |\mathbf{v}_i \cdot \mathbf{e}|.$$

于是，两节非相邻板凳不碰撞等价于至少存在一个候选方向使投影区间分离，即

$$\exists \mathbf{e} \in \mathcal{E}_{ij}, \quad |(P_i^c - P_j^c) \cdot \mathbf{e}| > \rho_i(\mathbf{e}) + \rho_j(\mathbf{e}).$$

反之，若对四个候选方向均有

$$|(P_i^c - P_j^c) \cdot \mathbf{e}| \leq \rho_i(\mathbf{e}) + \rho_j(\mathbf{e}), \quad \forall \mathbf{e} \in \mathcal{E}_{ij},$$

则两矩形在所有候选轴上的投影均重叠，判定为发生实体碰撞。由此，时刻 t 下的全队无碰撞约束可写为

$$\forall i, j, |i - j| > 1, \quad \exists \mathbf{e} \in \mathcal{E}_{ij}(t) \text{ 使上式严格分离成立.}$$

这就是分离轴定理在本题中的具体应用：先由把手位置确定每节板凳的矩形区域，再对所有非相邻矩形板凳逐对检查四个投影方向。

为了直观呈现“投影区间分离即不碰撞”的判定方式，分离轴定理在矩形板凳上的应用示意图如下。

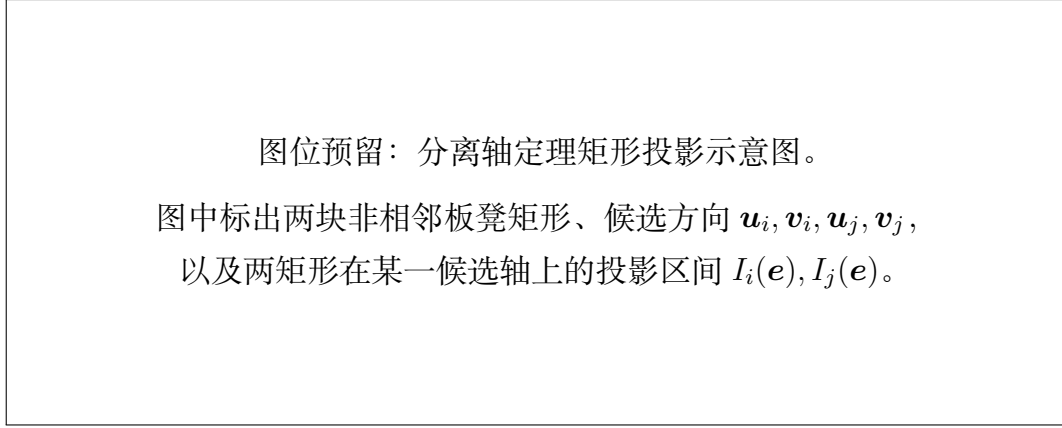


图 5 分离轴定理在矩形板凳碰撞判定中的应用示意

7.4 临界时刻与单调性

设

$$\chi(t) = \begin{cases} 0, & \text{时刻 } t \text{ 无碰撞,} \\ 1, & \text{时刻 } t \text{ 发生碰撞.} \end{cases}$$

则终止时刻可定义为

$$T_{\text{stop}} = \sup\{t : \chi(t) = 0\}.$$

在题设盘入过程中，龙头持续向内，板凳间距只会逐渐压缩，因此 $\chi(t)$ 在临界附近呈现从 0 到 1 的单调跃迁。这一结构意味着：只要找到一个无碰撞时刻和一个碰撞时刻，就能在两者之间逼近临界点。

因此，我们采用扫描加二分的方式确定首次碰撞时刻。首先按时间顺序扫描若干采样点，找到最后一个满足 $\chi(t) = 0$ 的时刻 t^- 和第一个满足 $\chi(t) = 1$ 的时刻 t^+ ，从而得到包含临界时刻的区间 $[t^-, t^+]$ 。随后取中点 $t_m = (t^- + t^+)/2$ 并重新计算 $\chi(t_m)$ ：若 $\chi(t_m) = 0$ ，则令 $t^- = t_m$ ；若 $\chi(t_m) = 1$ ，则令 $t^+ = t_m$ 。重复该过程，直到区间长度小于给定容差，此时 t^- 即作为无碰撞终止时刻的数值近似。

7.5 求解流程

Algorithm 2 问题二首次碰撞时刻搜索流程

Input: 问题一得到的把手轨迹，板凳宽度和把手间距。

Output: 无碰撞终止时刻 T_{stop} 及终止状态。

- 1: 根据把手坐标构造每节板凳对应的矩形区域 $\mathcal{R}_i(t)$ 。
 - 2: 对每个采样时刻 t ，用分离轴定理检测所有满足 $|i - j| > 1$ 的板凳对。
 - 3: 找到最后一个无碰撞采样时刻 t^- 和第一个发生碰撞采样时刻 t^+ 。
 - 4: 当 $t^+ - t^-$ 仍大于给定容差时，令 $t_m = (t^- + t^+)/2$ 并计算碰撞判定函数 $\chi(t_m)$ 。
 - 5: 根据 $\chi(t_m)$ 更新区间 $[t^-, t^+]$ ，直至区间长度满足精度要求。
 - 6: **return** T_{stop} 和终止状态。
-

7.6 结果解释

求得的无碰撞终止时刻为

$$T_{\text{stop}} = 412.474 \text{ s.}$$

最终二分区间宽度小于 $1.0 \times 10^{-8} \text{ s}$ 。首次发生碰撞的板对为 $[0, 8]$ 。这说明在终止时刻之后，头部与第 8 节板之间首先进入接触状态。

首次碰撞附近的最小投影裕度随时间变化如下。曲线由正变负的位置对应碰撞判定函数由 0 跃迁到 1 的临界区域，二分过程即在该变号区间内不断压缩终止时刻范围。

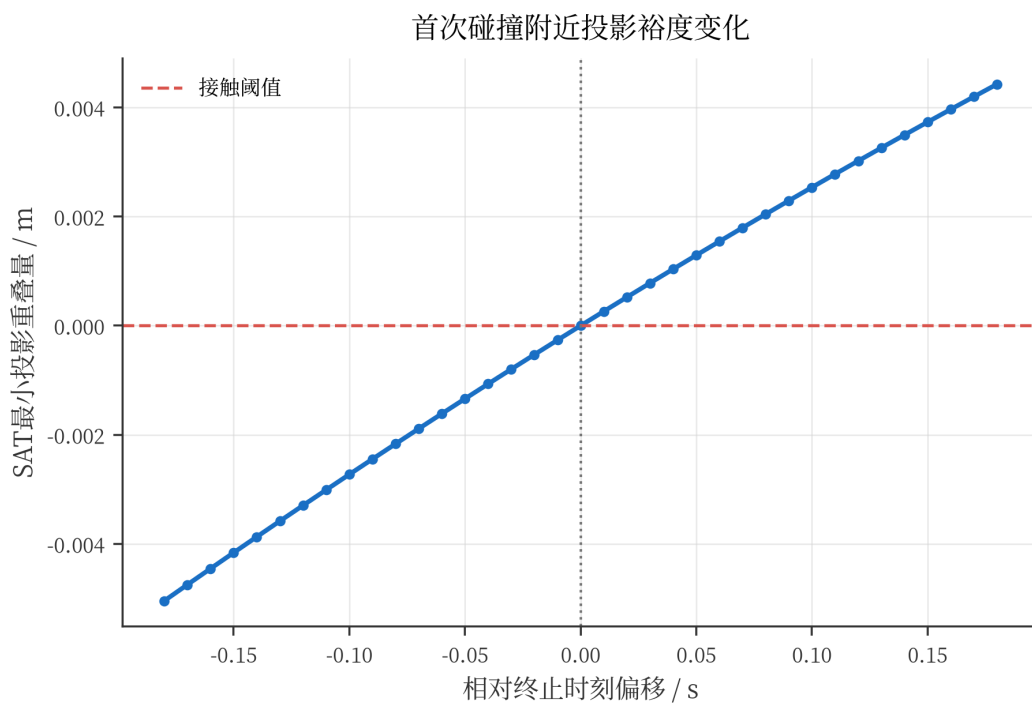


图 6 首次碰撞附近最小投影裕度变化

首次碰撞附近板凳龙的整体位置示意图如下，用于标明龙头与第 8 节板凳的相对位置关系。

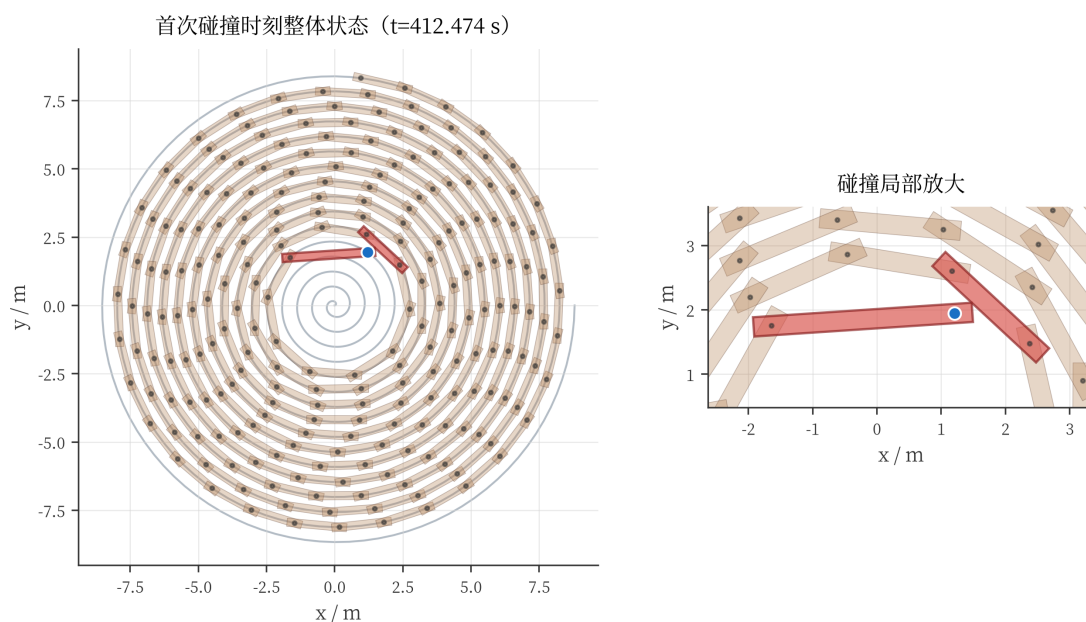


图 7 首次碰撞时刻附近板凳龙位置示意

终止状态已按附件二格式写入工作簿。该时刻全体把手速度范围为 0.973 到 1.000 m/s。

表 3 问题二终止状态抽样表

项目	x (m)	y (m)	速度 (m/s)
龙头	1.209931	1.942784	1.000000
第 1 节龙身	-1.643792	1.753399	0.991551
第 51 节龙身	1.281201	4.326588	0.976858
第 101 节龙身	-0.536246	-5.880138	0.974550
第 151 节龙身	0.968840	-6.957479	0.973608
第 201 节龙身	-7.893161	-1.230764	0.973096
龙尾 (后)	0.956217	8.322736	0.972938

7.7 小结

问题二的终止时刻求解可以概括为“定义碰撞判定函数 $\chi(t)$ ，定位其由 0 跃迁到 1 的临界点”。分离轴原理提供了矩形相交判定的数学基础，扫描与二分则提供了临界时刻的数值定位方式。该处理方式比仅凭中心线或经验距离判断更符合题目的实体碰撞要求。

八、问题三：最小螺距的确定

8.1 建模思路

问题三要求确定龙头前把手能够盘入半径为 4.5 m 的调头空间边界时所需的最小螺距。螺距越小，相邻圈螺线越密，龙头附近的板凳越容易与外圈板凳接近。因此，我们先以题目给定的 55 cm 作为初始螺距，并逐步减小螺距进行分析。

当螺距为 55 cm 时，根据前两问的计算过程，舞龙队在到达调头空间边界之前仍具有较大的空间余量。因此，最小螺距必然不大于 55 cm。接下来从该上界出发，按步长逐步缩小螺距；每给定一个螺距，就判断龙头前把手从初始位置盘入到调头空间边界的过程中是否出现接触风险。若没有出现接触风险，则继续减小螺距；若出现接触风险，则退回上一螺距，并缩小步长继续搜索，直到步长达到给定精度。

8.2 建立模型

沿用问题一中的等距螺线模型，设螺距为 d ，则盘入螺线可表示为

$$r(\theta) = \frac{d}{2\pi}\theta, \quad x(\theta) = r(\theta) \cos \theta, \quad y(\theta) = r(\theta) \sin \theta.$$

当龙头前把手到达调头空间边界时，有

$$r(\theta_0) = R, \quad R = 4.5 \text{ m},$$

即

$$\theta_0(d) = \frac{2\pi R}{d}.$$

龙头前把手到达调头空间边界时的几何关系示意图如下。图中边界圆与盘入螺线的交点对应 $\theta_0(d)$ ，也是本问判断“能否盘入调头空间”的终点状态。

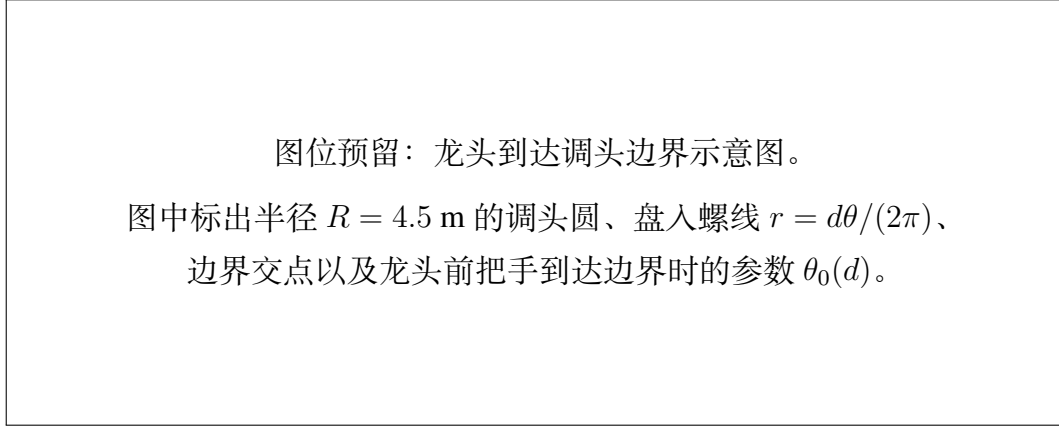


图 8 龙头前把手到达调头空间边界的几何示意

对于给定的 θ 与 d ，由相邻把手距离递推得到前若干把手坐标

$$P_k(\theta, d) = (x_k(\theta, d), y_k(\theta, d)), \quad k = 0, 1, \dots, 35.$$

其中，龙头板凳的把手间距取 $\ell_H = 2.86 \text{ m}$ ，其余板凳的把手间距取 $\ell_B = 1.65 \text{ m}$ 。递推时通过

$$\|P_{k+1}(\theta, d) - P_k(\theta, d)\|_2 = \ell_k$$

确定下一把手对应的角参数。

在临界碰撞判断中，主要考察龙头及其后一节板凳与前方若干龙身板凳之间的相对位置。对第 j 块板凳，其中心轴线由相邻把手 P_j, P_{j+1} 确定，记为

$$A_j x + B_j y + C_j = 0,$$

其中

$$A_j = y_{j+1} - y_j, \quad B_j = x_j - x_{j+1}, \quad C_j = -x_j y_{j+1} + x_{j+1} y_j.$$

龙头和第 1 节龙身的外侧角点由把手连线方向、法向方向以及板凳宽度确定。设这些角点构成集合 $\mathcal{V}(\theta, d)$ ，角点 $V = (x_V, y_V)$ 到第 j 块板凳轴线的距离为

$$\rho(V, L_j) = \frac{|A_j x_V + B_j y_V + C_j|}{\sqrt{A_j^2 + B_j^2}}.$$

取 $j = 3, \dots, 30$ 作为待检板凳范围，定义

$$D(\theta, d) = \min_{\substack{V \in \mathcal{V}(\theta, d) \\ 3 \leq j \leq 30}} \rho(V, L_j).$$

由于板凳宽度为 0.30 m，半宽为 0.15 m。当龙头附近外侧角点到其他板凳轴线的最小距离降至 0.15 m 时，可认为实体轮廓到达接触临界。因此，判定函数为

$$F(d) = \min_{\theta \in [\theta_0(d), \theta_0(d) + 6]} D(\theta, d) - 0.15.$$

若 $F(d) > 0$ ，说明在该螺距下未达到接触临界；若 $F(d) \leq 0$ ，说明该螺距下已经出现接触风险。

8.3 搜索最小螺距

为了搜索最小螺距，我们采用逐步逼近的方法。

Step1：确定初始螺距。取题设螺距 $d = 0.55$ m 作为初始值，并选取 0.01 m 作为初始步长。

Step2：判断是否发生碰撞。对当前螺距，计算龙头到达调头空间边界时的 $\theta_0(d)$ ，并在 $[\theta_0(d), \theta_0(d) + 6]$ 内检查龙头及第 1 节龙身与第 3 至第 30 节龙身之间的距离关系。

Step3：循环迭代。若未达到接触临界，则继续减小螺距；若达到接触临界，则返回上一螺距，并将步长缩小为原来的十分之一，继续判断。

Step4：终止。当步长缩小到 10^{-6} m 时停止循环，得到较为精确的最小螺距。

根据上述过程，得到不同搜索步长下的最小螺距近似值，如表4所示。

表 4 逐步逼近最小螺距

步长	螺距	单位
0.01	0.46	m
0.001	0.451	m
0.0001	0.4504	m
0.00001	0.45034	m
0.000001	0.450332	m

保留两位小数后，盘入调头空间所需的最小螺距为

$$d_{\min} = 0.45 \text{ m}.$$

在粗搜索阶段，螺距 $d = 0.45$ m 已触发接触风险，这说明最小可行螺距位于 0.45 m 的右侧；继续细化后得到上述结果。

九、问题四：调头路径的分段建模与状态递推

9.1 问题分解

问题四给出盘入螺距为 1.7 m，要求在半径 4.5 m 的调头空间内构造盘入与盘出之间的连接路径，并输出 -100 到 100 s 的运动状态。本问先由螺线与调头圆的交点确定调头入点，再按两段相切圆弧和半径比 2 : 1 建立调头路径，最后在该复合路径上递推全队把手状态。

9.2 调头曲线的几何关系

设盘入螺线与盘出螺线关于调头空间中心对称，中间连接段由两段相切圆弧组成，且满足

$$R_1 = 2R_2.$$

为便于表达，设盘入螺线与圆弧 1 的切点为 P_{in} ，两圆弧切点为 Q ，圆弧 2 与盘出螺线切点为 P_{out} 。那么整条路径由四段拼接而成：

盘入螺线 $\rightarrow$ 圆弧 1 $\rightarrow$ 圆弧 2 $\rightarrow$ 盘出螺线。

为了使曲线连续且光滑，相邻段在连接点处应满足位置连续与切向连续。记盘入段与盘出段在边界处的切向单位向量分别为 t_{in} 和 t_{out} ，则圆弧 1、圆弧 2 的圆心分别落在对应切点的法线方向上；再结合两圆弧相切与半径比约束，可确定圆心、半径和圆弧转角。

9.3 固定交点下的几何关系证明

设盘入螺距为 $p = 1.7$ m，令

$$b = \frac{p}{2\pi}, \quad r = b\theta.$$

调头空间为圆域

$$\Omega = \{(x, y) : x^2 + y^2 \leq R^2\}, \quad R = 4.5 \text{ m}.$$

盘入螺线第一次到达调头圆边界时满足

$$r(\theta_F) = R,$$

由此得到入点对应的极角

$$\theta_F = \frac{R}{b} = \frac{2\pi R}{p}.$$

对应的交点为

$$F = P_{\text{in}} = (R \cos \theta_F, R \sin \theta_F).$$

盘出螺线与盘入螺线关于调头空间中心成中心对称，因此出点为

$$P_{\text{out}} = -P_{\text{in}}.$$

固定交点情形下的调头曲线几何关系示意图如下。

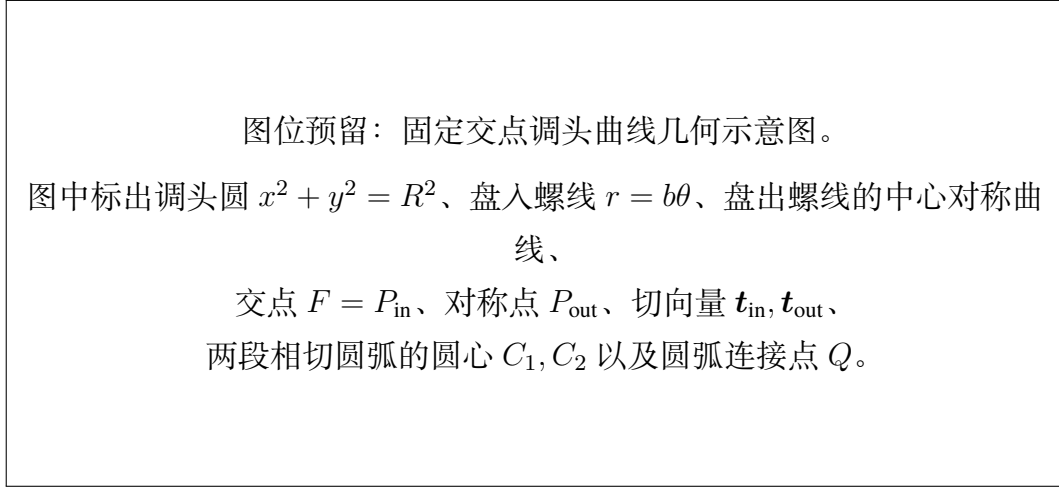


图 9 固定交点情形下调头路径的几何关系示意

由螺线参数方程

$$P(\theta) = b\theta(\cos \theta, \sin \theta)$$

可得切向量

$$P'(\theta) = b(\cos \theta - \theta \sin \theta, \sin \theta + \theta \cos \theta).$$

于是盘入端点的单位切向量为

$$\mathbf{t}_{\text{in}} = \frac{P'(\theta_F)}{\|P'(\theta_F)\|}.$$

记径向单位向量为

$$\mathbf{e}_r = (\cos \theta_F, \sin \theta_F).$$

设螺线切线与调头圆切线之间的夹角为 α ，则

$$\alpha = \frac{\pi}{2} - \arccos(\mathbf{t}_{\text{in}} \cdot \mathbf{e}_r).$$

将 $\mathbf{t}_{\text{in}}$ 代入上式，可得

$$\alpha = \frac{\pi}{2} - \arccos \frac{(\cos \theta_F - \theta_F \sin \theta_F) \cos \theta_F + (\sin \theta_F + \theta_F \cos \theta_F) \sin \theta_F}{\sqrt{\theta_F^2 + 1}} = \frac{\pi}{2} - \arccos \frac{1}{\sqrt{\theta_F^2 + 1}}.$$

两段圆弧与盘入、盘出螺线相切，且两段圆弧在连接点 Q 处相切。由图9中的直角三角关系，圆心连线对应的有效半径和为

$$R_1 + R_2 = \frac{R}{\cos \alpha}.$$

又由题设

$$R_1 = 2R_2,$$

解得

$$R_1 = \frac{2R}{3\cos\alpha}, \quad R_2 = \frac{R}{3\cos\alpha}.$$

连接点 Q 两侧的圆弧转角相同, 均为

$$\beta = \pi - 2\alpha.$$

于是调头曲线长度为

$$L_{\text{turn}} = R_1\beta + R_2\beta = (R_1 + R_2)(\pi - 2\alpha) = \frac{R}{\cos\alpha}(\pi - 2\alpha).$$

其中

$$\theta_F = \frac{2\pi R}{p}, \quad \alpha = \frac{\pi}{2} - \arccos \frac{1}{\sqrt{\theta_F^2 + 1}}.$$

代入 $p = 1.7 \text{ m}$ 、 $R = 4.5 \text{ m}$, 得到

$$\theta_F = 16.631961, \quad \alpha = 0.060053,$$

$$R_1 = 3.005418 \text{ m}, \quad R_2 = 1.502709 \text{ m},$$

$$L_{\text{turn}} = 13.621245 \text{ m}.$$

9.4 路径参数与求解流程

根据上述几何关系, 调头路径满足

$$\left\{ \begin{array}{l} \text{圆弧 1 与盘入螺线相切,} \\ \text{圆弧 1 与圆弧 2 相切,} \\ \text{圆弧 2 与盘出螺线相切,} \\ R_1 = 2R_2, \\ P_{\text{out}} = -P_{\text{in}}, \\ P_{\text{in}} = (R \cos \theta_F, R \sin \theta_F). \end{array} \right.$$

求解时先计算 θ_F 和 α , 再由半径比求 R_1, R_2 , 最后按盘入螺线、圆弧 1、圆弧 2、盘出螺线的顺序建立弧长参数函数。

Algorithm 3 问题四固定交点调头路径与状态求解流程

Input: 螺距 $p = 1.7$, 调头空间半径 $R = 4.5$, 整数时刻 $t \in [-100, 100]$ 。

Output: 调头路径参数、各时刻把手位置与速度。

- 1: 计算盘入螺线与调头圆交点对应的极角 $\theta_F = 2\pi R/p$ 。
 - 2: 由交点处螺线切向确定边界切线夹角 α 。
 - 3: 根据 $R_1 = 2R/(3 \cos \alpha)$ 和 $R_2 = R/(3 \cos \alpha)$ 确定两段圆弧半径。
 - 4: 按盘入螺线、圆弧 1、圆弧 2、中心对称盘出螺线的顺序建立复合弧长参数路径。
 - 5: 对 $t \in [-100, 100]$ 逐秒递推全部把手位置, 并由差分计算速度。
 - 6: **return** 调头路径参数和全队运动状态。
-

按上述参数得到的调头路径整体示意图如下, 用于展示盘入螺线、两段圆弧和盘出螺线的空间拼接关系。

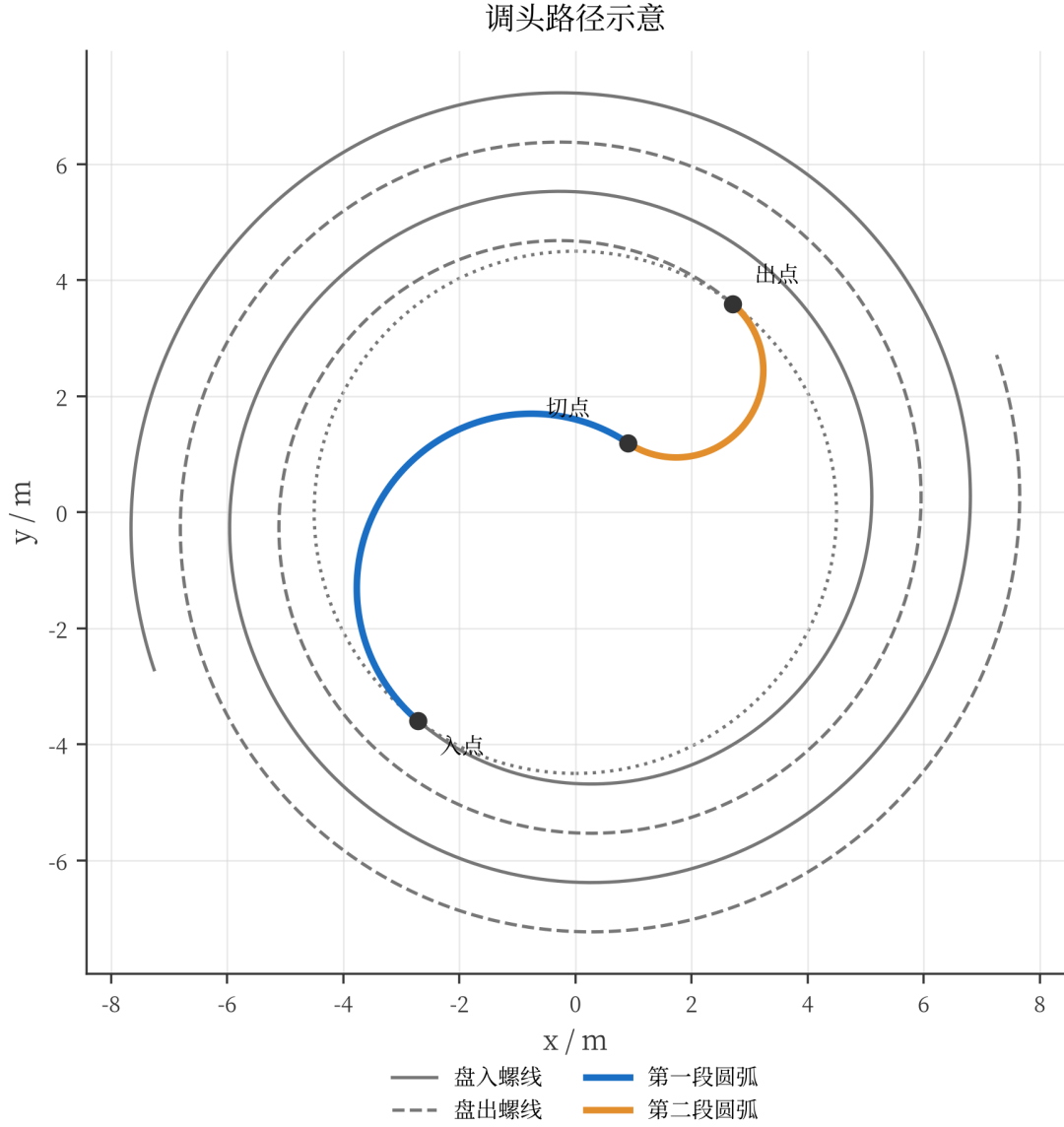


图 10 盘入、两段圆弧与盘出路径的整体示意

9.5 状态递推与结果

在路径确定后，龙头前把手沿该复合路径以 1 m/s 运动。后续把手仍由固定把手间距逐节递推得到。对于任意时刻 t ，将路径视为弧长参数 s 的函数 $P(s)$ ，先由头速确定龙头对应弧长，再由

$$\|P_{i+1}(s) - P_i(s)\|_2 = d_i$$

逐节向后求解即可。速度采用问题一中的中心差分形式从位置序列计算。

调头路径参数如下：

$$R_1 = 3.005418 \text{ m}, \quad R_2 = 1.502709 \text{ m},$$

$$L_{\text{turn}} = 13.621245 \text{ m}.$$

整段 -100 到 100 s 的运动状态按上述复合路径递推得到。完整位置表含 224 个把手在 201 个整数时刻的 x, y 坐标, 规模为 449×202 ; 完整速度表含 224 个把手在 201 个整数时刻的速度, 规模为 225×202 。两张表已写入结果工作簿 `result4.xlsx`, 表6和表7列出关键把手在部分时刻的抽样结果。

板凳龙在盘入盘出复合路径上的整体状态示意图如下, 用于展示路径改变后全队把手仍按固定距离逐节递推。

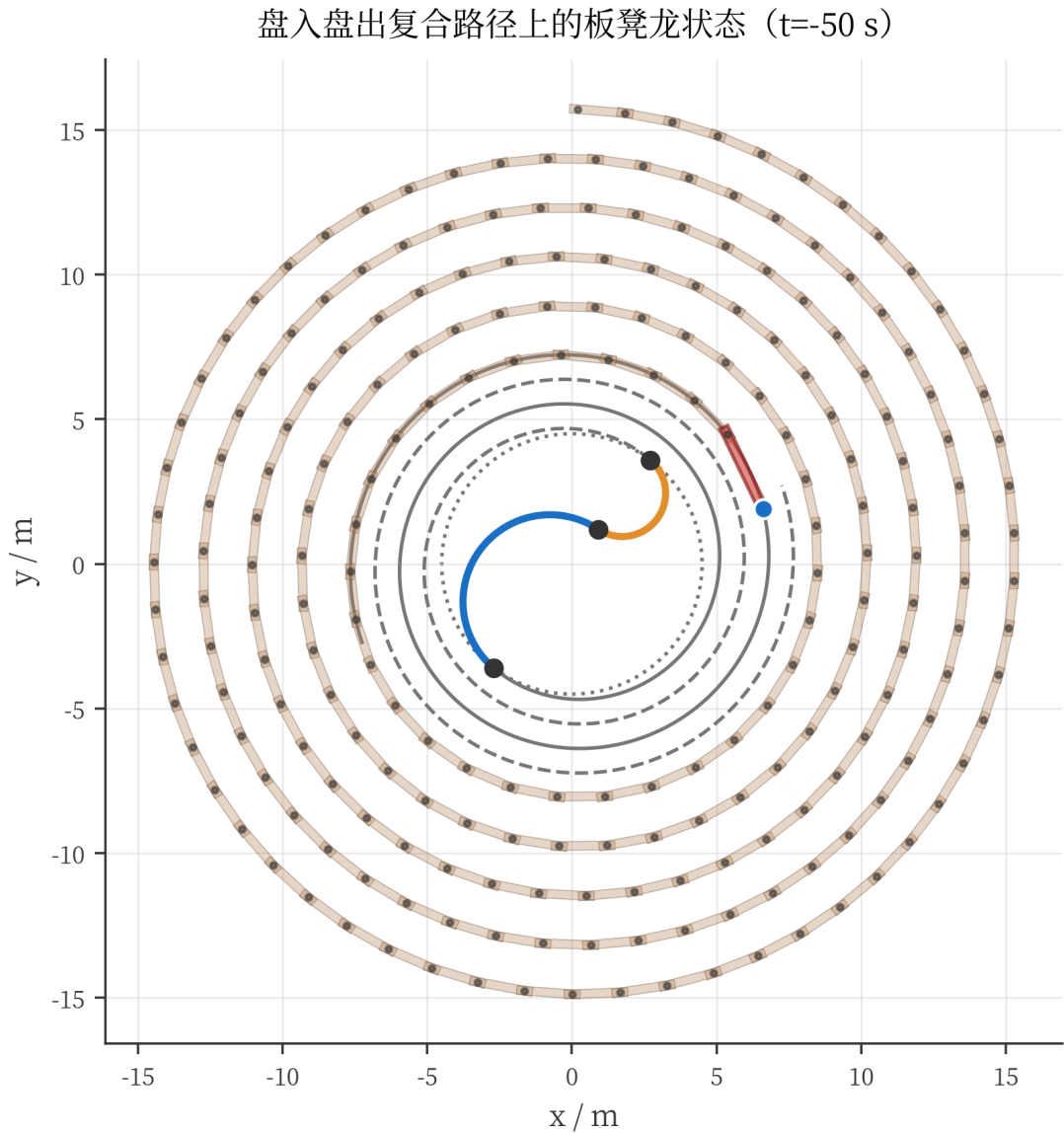


图 11 板凳龙在盘入盘出复合路径上的状态示意

当龙头位于圆弧调头段时的板凳龙状态示意图如下, 该图用于说明调头过程中仍采用同一套相邻把手距离递推约束。

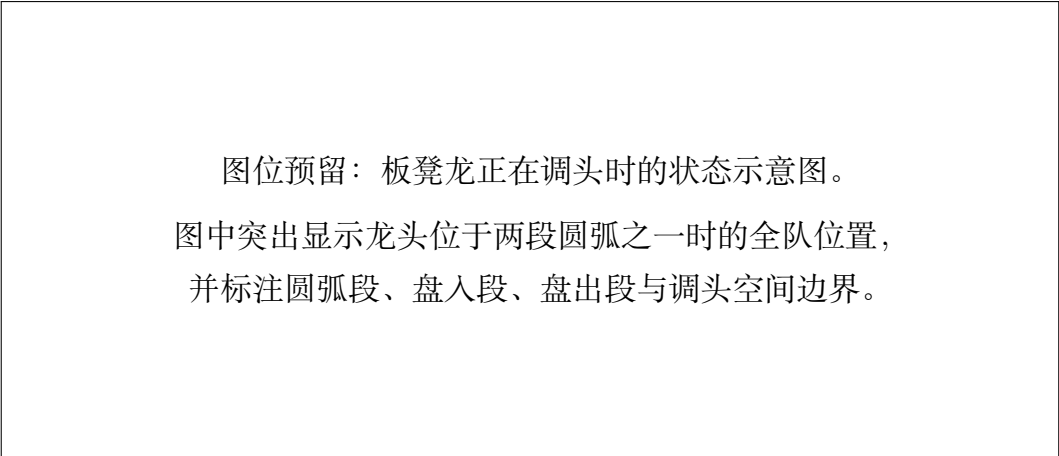


图 12 板凳龙处于圆弧调头段时的状态示意

表 5 问题四调头路径参数表

量	数值	单位
入点极角 θ_F	16.631961	rad
切线夹角 α	0.060053	rad
圆弧 1 半径 R_1	3.005418	m
圆弧 2 半径 R_2	1.502709	m
圆弧转角 β	3.021487	rad
调头路径长度 L_{turn}	13.621245	m

表 6 问题四关键把手位置抽样表

项目	-100 s	-50 s	0 s	50 s	100 s
龙头 x	7.778034	6.608301	-2.711856	1.332696	-3.157229
龙头 y	3.717164	1.898865	-3.591078	6.175324	7.548511
第 1 节 x	6.209273	5.366911	-0.063534	3.862265	-0.346890
第 1 节 y	6.108521	4.475403	-4.670888	4.840828	8.079166
第 51 节 x	-10.608038	-3.629945	2.459962	-1.671385	2.095033
第 51 节 y	2.831491	-8.963800	-7.778145	-6.076713	4.033787
第 101 节 x	-11.922761	10.125787	3.008493	-7.591816	-7.288774
第 101 节 y	-4.802378	-5.972247	10.108539	5.175487	2.063875
第 151 节 x	-14.351032	12.974784	-7.002789	-4.605165	9.462513
第 151 节 y	-1.980993	-3.810357	10.337482	-10.386988	-3.540357
第 201 节 x	-11.952942	10.522509	-6.872842	0.336952	8.524374
第 201 节 y	10.566998	-10.807425	12.382609	-13.177610	8.606933
龙尾 x	-1.011059	0.189809	-1.933627	5.859094	-10.980157
龙尾 y	-16.527573	15.720588	-14.713128	12.612894	-6.770006

表 7 问题四关键把手速度抽样表

项目	-100 s	-50 s	0 s	50 s	100 s
龙头	1.000000	1.000000	1.000000	1.000000	1.000000
第 1 节龙身	0.999904	0.999762	0.998687	1.000363	1.000124
第 51 节龙身	0.999346	0.998642	0.995134	0.949935	1.003966
第 101 节龙身	0.999091	0.998248	0.994448	0.948482	1.096263
第 151 节龙身	0.998944	0.998047	0.994156	0.948038	1.095306
第 201 节龙身	0.998849	0.997925	0.993994	0.947823	1.094933
龙尾（后）	0.998817	0.997885	0.993944	0.947760	1.094833

9.6 小结

Q4 的核心是在固定交点、两段相切圆弧和半径比 2 : 1 的约束下建立复合路径，再以该路径为基础计算前后运动状态。该路径下相邻把手距离最大误差为 1.33×10^{-9} m，说明逐节递推在调头前后仍保持了板凳连接约束。

十、问题五：最大恒定速度的求解

10.1 模型建立

问题五要求在问题四给定调头路径上，确定龙头前把手可采用的最大恒定速度，并保证任一把手速度均不超过 2 m/s。与前四问不同，本问不再改变几何路径，而是在既定路径上讨论速度约束；因此核心是把全队速度响应转化为龙头速度的比例放大问题。

设单位头速下第 i 个把手在时刻 t 的速度为 $v_i^{(1)}(t)$ 。当龙头恒定速度由 1 m/s 改为 v_H 时，路径形状和各把手间距约束不变，时间尺度按同一比例缩放，故各把手速度满足

$$v_i(t; v_H) = v_H v_i^{(1)}(t),$$

从而速度约束可写为

$$\max_{i,t} v_i(t; v_H) \leq 2.$$

于是最大头速满足

$$v_{\max} = \frac{2}{\max_{i,t} v_i^{(1)}(t)}.$$

该式把“所有把手速度均受限”的多变量约束，化为单位头速下最大速度放大倍数的计算。后续只需先求出速度矩阵 $\{v_i^{(1)}(t)\}$ ，再由其中的最大值反推龙头速度上界。

10.2 速度递推与约束转化

设第 $i-1$ 个把手与第 i 个把手之间的连杆方向单位向量为 $\mathbf{e}_i$ ，两把手所在路径处的切向单位向量分别为 $\boldsymbol{\tau}_{i-1}$ 和 $\boldsymbol{\tau}_i$ 。由相邻把手距离约束对时间求导，可得沿连杆方向的速度投影相等：

$$v_{i-1}(\boldsymbol{\tau}_{i-1} \cdot \mathbf{e}_i) = v_i(\boldsymbol{\tau}_i \cdot \mathbf{e}_i).$$

因此，在单位龙头速度 $v_0 = 1$ m/s 下，可逐节递推

$$v_i^{(1)}(t) = v_{i-1}^{(1)}(t) \left| \frac{\boldsymbol{\tau}_{i-1} \cdot \mathbf{e}_i}{\boldsymbol{\tau}_i \cdot \mathbf{e}_i} \right|.$$

对所有 $i = 0, 1, \dots, 223$ 和 $t = -100, -99, \dots, 100$ s 计算 $v_i^{(1)}(t)$ ，即可得到速度放大倍数

$$\lambda_{\max} = \max_{i,t} v_i^{(1)}(t).$$

为定位约束最紧的位置和时刻，进一步定义

$$M(t) = \max_i v_i^{(1)}(t), \quad G_i = \max_t v_i^{(1)}(t).$$

其中 $M(t)$ 反映每一时刻全队速度上界的变化， G_i 反映各把手在整个调头过程中的最大速度放大情况。前者用于定位限制时刻，后者用于解释限制把手来源。

基于 $M(t)$ 可绘制全队最大把手速度随时间变化图，用于说明速度约束如何在调头过程中形成。

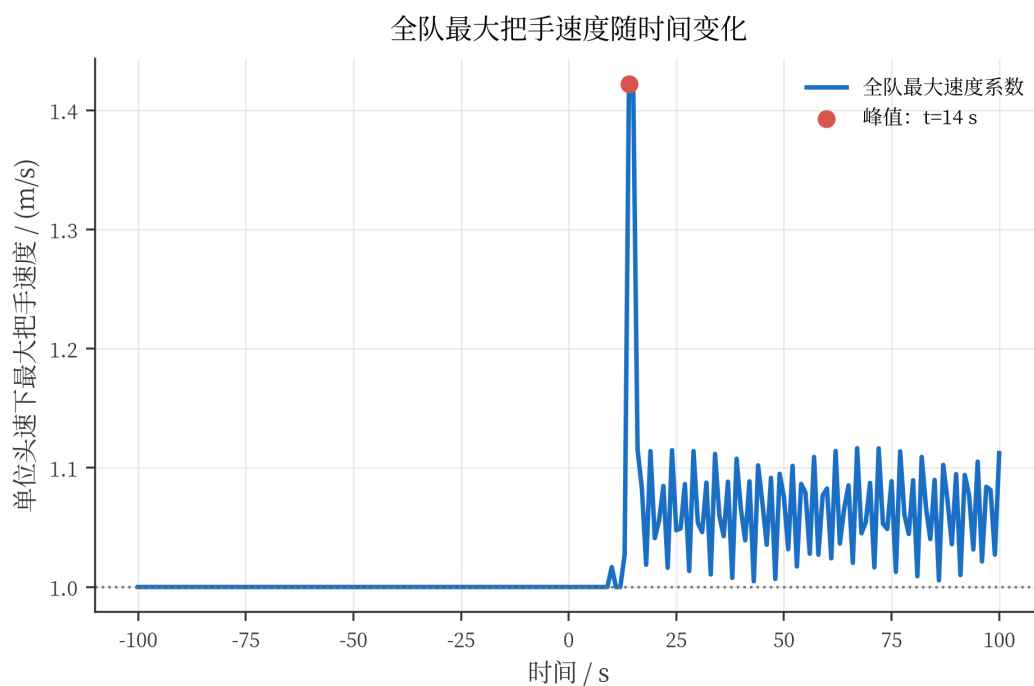


图 13 单位头速下全队最大把手速度随时间变化

进一步把速度放大倍数按“把手编号-时刻”展开，可得到单位头速下的速度放大倍数热力图，用于同时观察限制时刻与限制把手的位置。

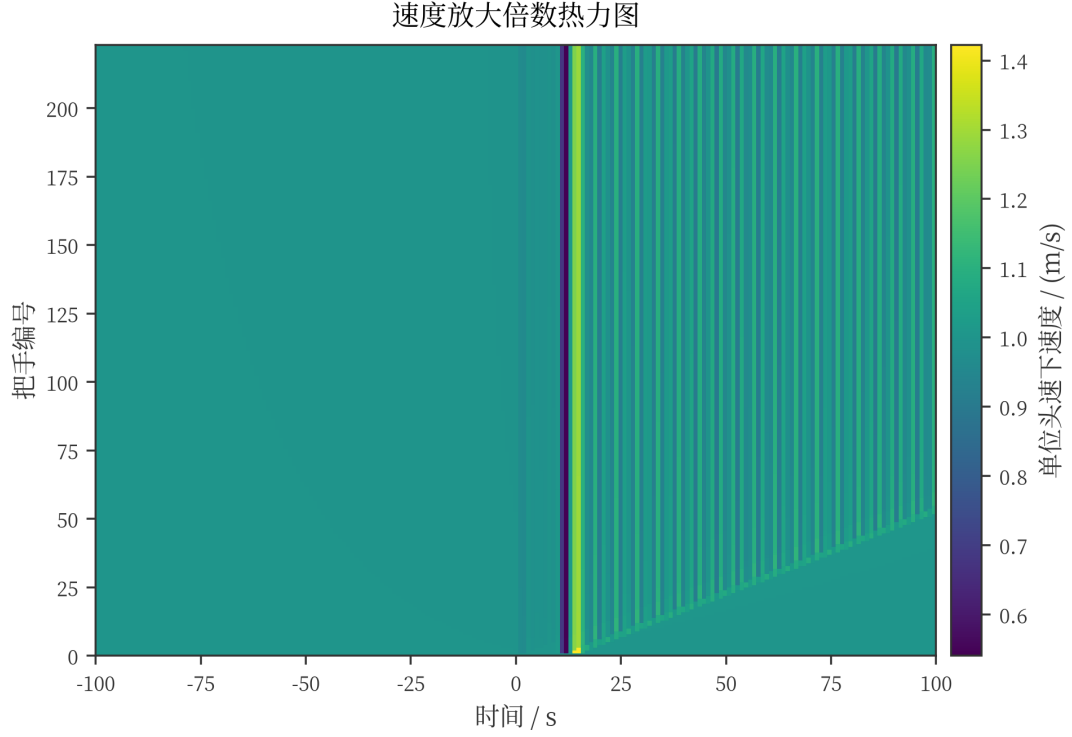


图 14 单位头速下速度放大倍数热力图

10.3 求解过程

求解时，首先读取问题四已经确定的固定交点复合路径；随后在 -100 到 100 s 的整数时刻上，按相邻把手距离约束递推全部把手位置；再根据切向投影关系计算单位头速下的速度矩阵。最后从该矩阵中提取 $\lambda_{\max}$ 、限制时刻和限制把手，并代入 $v_{\max} = 2/\lambda_{\max}$ 得到龙头速度上界。

Algorithm 4 问题五最大恒定头速反推流程

Input: 问题四固定调头路径，最大允许把手速度 2 m/s。

Output: 龙头最大恒定速度 $v_{\max}$ 。

- 1: 由固定交点和两段相切圆弧构造复合路径。
 - 2: 对每个整数时刻 $t \in [-100, 100]$ ，递推求解全部把手位置。
 - 3: 在单位头速下递推计算各把手速度。
 - 4: 取速度矩阵中的最大值作为 $\lambda_{\max}$ 。
 - 5: 根据 $v_{\max} = 2/\lambda_{\max}$ 反推龙头速度上界。
 - 6: **return** $v_{\max}$ 。
-

10.4 结果分析

最终得到的最大恒定头速为

$$v_{\max} = 1.406 \text{ m/s},$$

对应最大放大倍数为 1.422。在该速度下，全队最大把手速度为

$$2.000 \text{ m/s},$$

满足题目要求。

单位头速速度矩阵中的最大值出现在 $t = 14 \text{ s}$ ，对应限制把手为第 1 节龙身。也就是说，龙头速度继续增大时，该把手将最先达到 2 m/s 的速度上限，因此它决定了本问的最大恒定头速。

表 8 问题五速度上界结果表

量	数值	单位
最大恒定头速	1.406	m/s
最大放大倍数	1.422	—
限制时刻	14	s
限制把手	第 1 节龙身	—
缩放后最大把手速度	2.000	m/s

10.5 小结

问题五是一个在固定几何路径上的速度约束反推问题。通过单位头速下的速度矩阵，可以同时得到最大速度放大倍数、限制时刻和限制把手；再由 2 m/s 的速度上限反推龙头恒定速度。该处理方式与问题四的固定交点路径保持同一几何口径，也使最终速度上界具有明确的约束来源。

十一、结果讨论

表9汇总了五个问题的核心结果。可以看出，这组题目并不是五个彼此独立的数值任务，而是同一条几何主线上的连续推进：Q1 负责把“运动”变成可计算的参数递推，Q2 把“是否碰撞”变成矩形交叠判定，Q3 把“最小螺距”变成角点到板凳轴线距离的临界搜索，Q4 把“调头路径”变成固定交点下的两圆弧几何构造，Q5 则把“速度上限”变成比例缩放下的约束反推。

表 9 五个问题的核心结果汇总

问题	核心结果
Q1	0–300 s 全把手轨迹生成完成，最大相邻把手距离误差为 8.32×10^{-12} m
Q2	无碰撞终止时刻为 412.474 s，首个碰撞板对为 [0, 8]
Q3	最小螺距为 0.45 m，粗扫触发螺距为 $d = 0.45$ m
Q4	固定交点两圆弧调头路径长度为 13.621 m
Q5	固定交点路径下最大恒定速度为 1.406 m/s

从结果稳定性看，Q1、Q2、Q3 和 Q4 都具有明确的边界检验或区间逼近结构，因此数值结论较为稳定。Q5 建立在问题四固定路径的速度矩阵之上，其速度上界由单位头速下的最大速度放大倍数直接给出。

从方法一致性看，全文始终使用同一套把手间距和几何递推框架，这一点避免了“Q1 按中心线算、Q2 按实体算”造成的逻辑断裂。尤其是 Q3 和 Q5，都直接建立在前序问题给出的几何约束之上，因此它们的结论必须与前序问题保持同一解释口径。

十二、结论

本文围绕“板凳龙”运动过程建立了统一的几何建模框架。首先，利用等距螺线弧长参数化与逐节递推，完成了盘入阶段全把手状态的精确恢复；随后，引入有限宽度矩形碰撞模型，求得无碰撞终止时刻；进一步以螺线与调头圆的交点为入点，在调头空间内构造两段相切圆弧；最后通过速度比例缩放，得到满足全队速度约束的最大恒定龙头速度。

本文的主要结论可以概括为四点：其一，等距螺线加弧长递推能够稳定描述板凳龙的连续运动；其二，有限宽度矩形模型比中心线模型更适合判定盘入阶段的碰撞边界；其三，Q3 在角点到板凳轴线距离判据下给出的最小螺距约为 0.45 m；其四，固定交点调头路径下的龙头最大恒定速度为 1.406 m/s。

因此，本文给出的不是一组孤立数值，而是一套可复用的建模流程：先定路径，再定约束，最后在统一几何框架下完成搜索、判定与反推。这个流程与题目要求是一一对应的，也为后续类似板式编队运动问题提供了可迁移的思路。

参考文献

- [1] 全国大学生数学建模竞赛组委会. 2024 高教社杯全国大学生数学建模竞赛 A 题：板凳龙闹元宵及附件 [Z]. 2024.

- [2] latexstudio. CUMCMThesis: 全国大学生数学建模竞赛 $\text{\LaTeX}$ 论文模板 [EB/OL]. <https://github.com/latexstudio/CUMCMThesis>.
- [3] Ericson C. Real-Time Collision Detection[M]. Boca Raton: CRC Press, 2005.
- [4] Burden R L, Faires J D, Burden A M. Numerical Analysis[M]. 10th ed. Boston: Cengage Learning, 2015.

附录 A 主要程序清单

本文附录主要代码采用 `ref_code/` 目录下的可读 Python 脚本。该目录中部分以 `.py` 结尾的文件实际为 Excel 或压缩文件，故不作为源码插入；用于复现实验口径的主要脚本如表10所示。

表 10 主要程序文件说明

对应内容	程序文件	作用说明
运动递推与问题一表格	<code>ref_code/24A_Q1.py</code>	龙头角度反解、逐节把手位置递推和速度输出
碰撞搜索与问题二表格	<code>ref_code/24A_Q2.py</code>	板凳几何关系、碰撞距离判定和终止状态输出
螺距搜索	<code>ref_code/24A_Q3.py</code>	掉头边界约束下的螺距可行性判断与二分搜索
调头与速度约束	<code>ref_code/24A_Q4.py</code>	双圆弧调头路径、盘入盘出状态和速度约束计算
补充参考程序	<code>ref_code/solution3.py</code>	参考代码中的补充求解程序

附录 B 主要代码

以下给出 `ref_code/` 目录中可读的主要 Python 代码。代码清单采用 CUMCMThesis 模板附录中的 `listings` 格式排版。

2.1 运动递推与问题一表格程序

```

1 import pylab as plt
2 import numpy as np
3 from numpy.ma.core import transpose
4 from scipy.optimize import fsolve

```

```

5 from numpy.linalg import norm
6 from scipy.interpolate import interp2d
7 from scipy.optimize import root_scalar
8 import pandas as pd
9 import time
10
11
12 def time2theta_head(v,d,time_span):
13     def equation(theta,time):
14         equation = (16 * np.pi * np.sqrt((32 * np.pi) ** 2 + 1) + 1 / 2 * np.log(
15             32 * np.pi + np.sqrt((32 * np.pi) ** 2 + 1))) - (theta / 2 * np.sqrt(theta ** 2 + 1) + 1
16             / 2 * np.log(
17                 theta + np.sqrt(theta ** 2 + 1))) - v * time/d*2*np.pi
18         return equation
19     theta_head=np.zeros(len(time_span))
20     for i in range(len(time_span)):
21         theta0=0
22         if i!=0:
23             theta1=theta_head[i-1]
24         else:
25             theta1 = 33 * np.pi
26         while theta1-theta0>1e-7:
27             theta_mid=(theta1+theta0)/2
28             if equation(theta_mid,time_span[i])<0:
29                 theta1=theta_mid
30             else:
31                 theta0=theta_mid
32             theta_head[i]=(theta1+theta0)/2
33     return theta_head
34
35 def solve_theta(theta_pre, board, d):
36     def g(theta):
37         return theta_pre**2 + theta**2 - (4*np.pi**2 * board**2) / d**2 - 2*theta*theta_pre*np.cos(
38             theta-theta_pre)
39     # 区间
40     theta0 = theta_pre
41     theta1 = theta_pre +np.arccos((d**2*theta_pre**2-2*np.pi**2*board**2)/(d**2*theta_pre**2))
42
43     sol = root_scalar(g, bracket=[theta0, theta1], method='bisect')
44     if sol.converged:
45         return sol.root
46     else:
47         raise ValueError("Root finding did not converge")
48
49 if __name__ == '__main__':
50     """
51     数据设置
52     """
53
54     run_time1=time.time()
55
56     #时间点
57     time=300
58     time=np.array([time])

```

```

59 #时间范围
60 time_start=0
61 time_end=300
62 dt=1
63 time_span=np.arange(0,time_end,dt)
64 if time_span[-1] < time_end:
65     time_span = np.append(time_span, time_end)
66 Nt=len(time_span)
67
68 # 螺距
69 d = 0.55
70 #速度
71 v=1.0
72 #把手个数
73 N_hand=224
74 #把手距离
75 hand_length=1.65*np.ones(N_hand-1)
76 hand_length[0]=2.86
77 #前距与边距
78 p=0.275
79 q=0.15
80 """
81 绘制螺线
82 """
83 alpha=np.linspace(0,32*np.pi,1000)
84 r=d/(2*np.pi)*alpha
85 plt.figure( dpi=180)
86 plt.plot(r*np.cos(alpha),r*np.sin(alpha),lw=0.5)
87
88 plt.axis('equal')
89
90
91
92 """
93 龙头时间——角度关系
94 """
95 thetas_head=time2theta_head(v,d,time_span)
96 theta_head_time=time2theta_head(v,d,time)
97 """
98 龙身角度递推
99 """
100 thetas=np.zeros((Nt,N_hand))
101 theta_time=np.zeros(N_hand)
102 thetas[:,0]=thetas_head
103 theta_time[0]=theta_head_time[0]
104 for i in range(N_hand-1):
105     theta_time[i + 1] = solve_theta(theta_time[i], hand_length[i], d)
106     for j in range(Nt):
107         thetas[j,i+1]=solve_theta(thetas[j,i],hand_length[i],d)
108
109
110
111 xs=d/(2*np.pi)*thetas*np.cos(thetas)
112 ys=d/(2*np.pi)*thetas*np.sin(thetas)
113
114

```

```

115 x=d/(2*np.pi)*theta_time*np.cos(theta_time)
116 y=d/(2*np.pi)*theta_time*np.sin(theta_time)
117
118 plt.plot(x, y,lw=0.5)
119
120
121 vec1 = np.vstack((x[:-1] - x[1:], y[:-1] - y[1:]))
122 vec_n = vec1 /np.linalg.norm(vec1, axis=0)
123 vec_t = np.vstack((-vec_n[1, :], vec_n[0, :]))
124
125 points_out1=np.vstack((x[:-1], y[:-1]))+p*vec_n+q*vec_t
126 points_out2=np.vstack((x[1:], y[1:]))-p*vec_n+q*vec_t
127 points_in1 = np.vstack((x[:-1], y[:-1])) + p * vec_n - q * vec_t
128 points_in2 = np.vstack((x[1:], y[1:])) - p * vec_n - q * vec_t
129
130 for i in range(N_hand - 1):
131     # 四边形顶点按顺序连接
132     x_poly = [points_out1[0, i], points_out2[0, i], points_in2[0, i], points_in1[0, i],
133               points_out1[0, i]]
134     y_poly = [points_out1[1, i], points_out2[1, i], points_in2[1, i], points_in1[1, i],
135               points_out1[1, i]]
136
137     plt.plot(x_poly, y_poly, 'k-',lw=0.5)
138     # 填充棕色内部
139     plt.fill(x_poly, y_poly, color='brown', alpha=0.5)
140
141 vs=np.zeros((Nt,N_hand))
142 vs[:, 0]=v
143
144 vec2 = np.dstack((xs[:, :-1] - xs[:, 1:], ys[:, :-1] - ys[:, 1:]))
145 vec_ns = vec2 / np.linalg.norm(vec2, axis=-1, keepdims=True)
146 vec3 = np.dstack((np.cos(thetas)-thetas*np.sin(thetas), np.sin(thetas)+thetas*np.cos(thetas)))
147 vec_ps = vec3 /np.linalg.norm(vec3,axis=-1,keepdims=True)
148
149
150 num = np.sum(vec_ps[:, :-1, :] * vec_ns, axis=-1) # (300, 233)
151 den = np.sum(vec_ps[:, 1:, :] * vec_ns, axis=-1) # (300, 233)
152 rate = num / den # (300, 233)
153
154 for j in range(1, N_hand):
155     vs[:, j] = vs[:, j - 1] * rate[:, j - 1]
156
157
158 xy = []
159 xs=transpose(xs)
160 ys=transpose(ys)
161 for i in range(xs.shape[0]):
162     xy.append(xs[i]) # 第 i 行 xs
163     xy.append(ys[i]) # 第 i 行 ys
164
165
166 df_xy=pd.DataFrame(xy)
167 df_xy.to_excel("Q1_xs_ys.xlsx", index=False, header=False)
168
169 vs=transpose(vs)
170 df_vs = pd.DataFrame(vs)

```

```

169 df_vs.to_excel("Q1_vs.xlsx", index=False, header=False)
170
171 plt.show()

```

Listing 1: 运动递推与问题一表格程序

2.2 碰撞搜索与问题二表格程序

```

1 import pylab as plt
2 import numpy as np
3 from numpy.ma.core import transpose
4 import pandas as pd
5 from scipy.interpolate import interp1d
6 from scipy.optimize import fsolve
7 """
8 数据设置
9 """
10 # 螺距
11 d = 0.55
12 # 速度
13 v = 1.0
14 # 把手个数
15 N_hand = 224
16 # 把手距离
17 hand_length = 1.65 * np.ones(N_hand - 1)
18 hand_length[0] = 2.86
19 # 前距与边距
20 p = 0.275
21 q = 0.15
22
23 def get_theta_head(t):
24     def equation(theta,t):
25         equation = (16 * np.pi * np.sqrt((32 * np.pi) ** 2 + 1) + 1 / 2 * np.log(
26             32 * np.pi + np.sqrt((32 * np.pi) ** 2 + 1))) - (theta / 2 * np.sqrt(theta ** 2 + 1) + 1
27             / 2 * np.log(
28                 theta + np.sqrt(theta ** 2 + 1))) - v * t/d*2*np.pi
29         return equation
30     theta0 = inverse(t)
31     sol = fsolve(equation, x0=theta0, args=(t,))
32     theta_head=sol[0]
33     return theta_head
34
35 def inverse(t):
36     def F(theta):
37         def func(theta):
38             return d/(2*np.pi) * (0.5 * theta * np.sqrt(theta ** 2 + 1) + 0.5 * np.log(theta + np.
39                 sqrt(theta ** 2 + 1)))
40
41         return (func(32 * np.pi) - func(theta))/v
42
43     theta=np.linspace(0,32*np.pi,1000)
44     F_val=F(theta)
45
46     F_inv_approx=interp1d(F_val,theta,kind='linear',fill_value='extrapolate')

```

```

45     theta0=F_inv_approx(t)
46     return theta0
47
48 def bitsection(f,a,b):
49     fa=f(a)
50     fb=f(b)
51     tol=1e-10
52
53     if fa*fb>0:
54         raise ValueError("端点值未异号")
55
56     for _ in range(100):
57         mid = (a + b) / 2
58         fmid = f(mid)
59         if abs(fmid)<tol:
60             return mid
61         elif fa*fmid<0:
62             b=mid
63             fb=fmid
64         else:
65             a=mid
66             fa=fmid
67     raise RuntimeError("函数未收敛")
68
69 def get_theta(theta_pre, board):
70     def g(theta):
71         return theta_pre**2 + theta**2 - (4*np.pi**2 * board**2) / d**2 - 2*theta*theta_pre*np.cos(
72             theta-theta_pre)
73
74     # 区间
75     theta0 = theta_pre + 0.001
76     theta1 = theta_pre + np.arccos(1-(2*np.pi**2*board**2)/(d**2*theta0**2))
77
78     sol = bitsection(g, theta0, theta1)
79
80     return sol
81
82 def distance(t,v,d,N_hand,hand_length,p,q):
83
84     """
85     龙头时间——角度关系
86     """
87     theta_head = get_theta_head(t)
88
89     """
90     龙身角度递推
91     """
92     thetas = np.zeros(N_hand)
93     thetas[0] = theta_head
94     for i in range(N_hand - 1):
95         thetas[i + 1] = get_theta(thetas[i], hand_length[i])
96
97     """
98     龙身坐标
99     """

```

```

100 x = d / (2 * np.pi) * thetas * np.cos(thetas)
101 y = d / (2 * np.pi) * thetas * np.sin(thetas)
102
103 """
104 法向量和切向量
105 """
106 vec1 = np.vstack((x[:-1] - x[1:], y[:-1] - y[1:]))
107 vec_n = vec1 / np.linalg.norm(vec1, axis=0)
108 vec_t = np.vstack((-vec_n[1, :], vec_n[0, :]))
109
110 """
111 四个角点坐标
112 """
113 points_out1 = np.vstack((x[:-1], y[:-1])) + p * vec_n + q * vec_t
114 points_out2 = np.vstack((x[1:], y[1:])) - p * vec_n + q * vec_t
115
116
117 A = y[1:] - y[:-1]
118 B = x[:-1] - x[1:]
119 C = -x[:-1] * y[1:] + x[1:] * y[:-1]
120
121 den = np.sqrt(A ** 2 + B ** 2)
122 distance = 100
123 for i in range(3, 30):
124     d1 = abs(A[i] * points_out1[0, 0] + B[i] * points_out1[1, 0] + C[i]) / den[i]
125     d2 = abs(A[i] * points_out2[0, 0] + B[i] * points_out2[1, 0] + C[i]) / den[i]
126     d3 = abs(A[i + 1] * points_out1[0, 1] + B[i + 1] * points_out1[1, 1] + C[i + 1]) / den[i +
127     1]
128     d4 = abs(A[i + 1] * points_out2[0, 1] + B[i + 1] * points_out2[1, 1] + C[i + 1]) / den[i +
129     1]
130     if min(d1, d2, d3, d4) < distance:
131         distance = min(d1, d2, d3, d4)
132
133 return distance
134
135 if __name__ == '__main__':
136
137     time=410
138     dt=0.01
139     while dt>1e-7:
140         if distance(time,v,d,N_hand,hand_length,p,q)>0.15:
141             time=time+dt
142         else:
143             time = time - dt
144             dt=dt/2
145     print("终止时刻",time)
146     print("距离",distance(331.7825199903101,v,0.55,N_hand,hand_length,p,q))
147
148 """
149 龙头时间——角度关系
150 """
151 theta_head = get_theta_head(time)
152
153 """
154 龙身角度递推

```

```

154 """
155 thetas = np.zeros(N_hand)
156 thetas[0] = theta_head
157 for i in range(N_hand - 1):
158     thetas[i + 1] = get_theta(thetas[i], hand_length[i])
159
160 """
161 龙身坐标
162 """
163 x = d / (2 * np.pi) * thetas * np.cos(thetas)
164 y = d / (2 * np.pi) * thetas * np.sin(thetas)
165
166 """
167 法向量和切向量
168 """
169 vec1 = np.vstack((x[:-1] - x[1:], y[:-1] - y[1:]))
170 vec_n = vec1 / np.linalg.norm(vec1, axis=0)
171 vec_t = np.vstack((-vec_n[1, :], vec_n[0, :]))
172
173 """
174 四个角点坐标
175 """
176 points_out1 = np.vstack((x[:-1], y[:-1])) + p * vec_n + q * vec_t
177 points_out2 = np.vstack((x[1:], y[1:])) - p * vec_n + q * vec_t
178 points_in1 = np.vstack((x[:-1], y[:-1])) + p * vec_n - q * vec_t
179 points_in2 = np.vstack((x[1:], y[1:])) - p * vec_n - q * vec_t
180
181 """
182 速度递推
183 """
184 vs=np.zeros(N_hand)
185 vs[0]=v
186 vec2 = np.column_stack((x[:-1] - x[1:], y[:-1] - y[1:]))
187 vec_ns = vec2 / np.linalg.norm(vec2, axis=1, keepdims=True)
188 vec3 = np.column_stack((np.cos(thetas) - thetas * np.sin(thetas), np.sin(thetas) + thetas * np.
189     cos(thetas)))
190 vec_ps = vec3 / np.linalg.norm(vec3, axis=1, keepdims=True)
191
192 rate = np.sum(vec_ps[:, -1, :] * vec_ns, axis=1) / np.sum(vec_ps[:, 1:, :] * vec_ns, axis=1)
193
194 for j in range(1, N_hand):
195     vs[j] = vs[j - 1] * rate[j - 1]
196
197 xy=np.vstack((x,y))
198 xy = transpose(xy)
199 df_xy = pd.DataFrame(xy)
200 df_xy.to_excel("Q2_xs_ys.xlsx", index=False, header=False)
201
202 vs = transpose(vs)
203 df_vs = pd.DataFrame(vs)
204 df_vs.to_excel("Q2_vs.xlsx", index=False, header=False)
205
206 """
207 绘制
208 """
209 alpha = np.linspace(0, 32 * np.pi, 1000)

```

```

209     r = d / (2 * np.pi) * alpha
210     plt.plot(r * np.cos(alpha), r * np.sin(alpha), lw=0.5)
211     #plt.plot(4.5 * np.cos(alpha), 4.5 * np.sin(alpha))
212     plt.axis('equal')
213
214     plt.plot(x, y, lw=0.5)
215
216     for i in range(N_hand - 1):
217         # 四边形顶点按顺序连接
218         x_poly = [points_out1[0, i], points_out2[0, i], points_in2[0, i], points_in1[0, i],
219                   points_out1[0, i]]
220         y_poly = [points_out1[1, i], points_out2[1, i], points_in2[1, i], points_in1[1, i],
221                   points_out1[1, i]]
222
223         plt.plot(x_poly, y_poly, 'k-', lw=0.5)
224         # 填充棕色内部
225         plt.fill(x_poly, y_poly, color='brown', alpha=0.5)
226
227     plt.show()

```

Listing 2: 碰撞搜索与问题二表格程序

2.3 螺距搜索程序

```

1 import numpy as np
2 from scipy.interpolate import interp1d
3 from scipy.optimize import fsolve
4 from scipy.optimize import root_scalar
5 """
6 数据设置
7 """
8 # 速度
9 v = 1.0
10 # 把手个数
11 N_hand = 224
12 # 把手距离
13 hand_length = 1.65 * np.ones(N_hand - 1)
14 hand_length[0] = 2.86
15 # 前距与边距
16 p = 0.275
17 q = 0.15
18 # 掉头边界
19 R=4.5
20
21 def get_theta_head(t,d):
22     def equation(theta,t):
23         equation = (16 * np.pi * np.sqrt((32 * np.pi) ** 2 + 1) + 1 / 2 * np.log(
24             32 * np.pi + np.sqrt((32 * np.pi) ** 2 + 1))) - (theta / 2 * np.sqrt(theta ** 2 + 1) + 1
25             / 2 * np.log(
26                 theta + np.sqrt(theta ** 2 + 1))) - v * t/d*2*np.pi
27         return equation
28     theta0 = inverse(t,d)
29     sol = fsolve(equation, x0=theta0, args=(t,))
30     theta_head=sol[0]

```

```

30     return theta_head
31
32 def get_t(theta,d):
33     def func(theta):
34         return d / (2 * np.pi) * (0.5 * theta * np.sqrt(theta ** 2 + 1) + 0.5 * np.log(theta + np.
35             sqrt(theta ** 2 + 1)))
36     return (func(32 * np.pi) - func(theta)) / v
37
38 def inverse(t,d):
39     def F(theta):
40         def func(theta):
41             return d/(2*np.pi) * (0.5 * theta * np.sqrt(theta ** 2 + 1) + 0.5 * np.log(theta + np.
42                 sqrt(theta ** 2 + 1)))
43         return (func(32 * np.pi) - func(theta))/v
44
45     theta=np.linspace(0,32*np.pi,1000)
46     F_val=F(theta)
47
48     F_inv_approx=interp1d(F_val,theta,kind='linear',fill_value='extrapolate')
49     theta0=F_inv_approx(t)
50     return theta0
51
52 def bitsection(f,a,b):
53     fa=f(a)
54     fb=f(b)
55     tol=1e-10
56
57     if fa*fb>0:
58         raise ValueError("端点值未异号")
59
60     for _ in range(100):
61         mid = (a + b) / 2
62         fmid = f(mid)
63         if abs(fmid)<tol:
64             return mid
65         elif fa*fmid<0:
66             b=mid
67             fb=fmid
68         else:
69             a=mid
70             fa=fmid
71     raise RuntimeError("函数未收敛")
72
73 def get_theta(theta_pre, board,d):
74     def g(theta):
75         return theta_pre**2 + theta**2 - (4*np.pi**2 * board**2) / d**2 - 2*theta*theta_pre*np.cos(
76             theta-theta_pre)
77
78     # 区间
79     theta0 = theta_pre + 0.001
80     theta1 = theta_pre + np.arccos(1-(2*np.pi**2*board**2)/(d**2*theta0**2))
81
82     sol = bitsection(g, theta0, theta1)
83
84     return sol

```

```

83
84
85 def distance(theta,d):
86
87     """
88     龙头时间——角度关系
89     """
90     theta_head = theta
91
92     """
93     龙身角度递推
94     """
95     thetas = np.zeros(36)
96     thetas[0] = theta_head
97     for i in range(35):
98         thetas[i + 1] = get_theta(thetas[i], hand_length[i],d)
99
100     """
101     龙身坐标
102     """
103     x = d / (2 * np.pi) * thetas * np.cos(thetas)
104     y = d / (2 * np.pi) * thetas * np.sin(thetas)
105
106     """
107     法向量和切向量
108     """
109     vec1 = np.vstack((x[:-1] - x[1:], y[:-1] - y[1:]))
110     vec_n = vec1 / np.linalg.norm(vec1, axis=0)
111     vec_t = np.vstack((-vec_n[1, :], vec_n[0, :]))
112
113     """
114     角点坐标
115     """
116     points_out1 = np.vstack((x[0:2], y[0:2])) + p * vec_n[:, 0:2] + q * vec_t[:,0:2]
117     points_out2 = np.vstack((x[1:3], y[1:3])) - p * vec_n[:, 0:2] + q * vec_t[:,0:2]
118
119
120     A = y[1:] - y[:-1]
121     B = x[:-1] - x[1:]
122     C = -x[:-1] * y[1:] + x[1:] * y[:-1]
123
124     den = np.sqrt(A ** 2 + B ** 2)
125     distance = 100
126     for i in range(3, 30):
127         d1 = abs(A[i] * points_out1[0, 0] + B[i] * points_out1[1, 0] + C[i]) / den[i]
128         d2 = abs(A[i] * points_out2[0, 0] + B[i] * points_out2[1, 0] + C[i]) / den[i]
129         d3 = abs(A[i + 1] * points_out1[0, 1] + B[i + 1] * points_out1[1, 1] + C[i + 1]) / den[i +
130         1]
131         d4 = abs(A[i + 1] * points_out2[0, 1] + B[i + 1] * points_out2[1, 1] + C[i + 1]) / den[i +
132         1]
133         if min(d1, d2, d3, d4) < distance:
134             distance = min(d1, d2, d3, d4)
135
136     return distance

```

```

137 def crush_judge(theta,d):
138     if distance(theta,d)-0.15<1e-6:
139         return 1
140     else:
141         return 0
142
143 def limit_theta(d):
144     theta0 = 2 * np.pi * R / d
145     return theta0
146
147
148
149
150
151 if __name__ == '__main__':
152
153     for d in np.arange(0.55, 0.4, -0.005):
154         theta0 = limit_theta(d)
155         for theta in np.arange(theta0+6, theta0, -0.05):
156             flag=crush_judge(theta,d)
157             if flag:
158                 break
159         if flag:
160             break
161
162
163     def min_distance(d):
164         min_dis=100
165         theta0 = limit_theta(d)
166         theta_span=np.linspace(theta0+6, theta0, 100)
167         for theta in theta_span:
168             dis=distance(theta,d)
169             if dis<min_dis:
170                 min_dis=dis
171         return min_dis
172     def F(d):
173         return min_distance(d)-0.15
174
175     sol=root_scalar(F,bracket=[d-0.02,d+0.02],method='bisect')
176     d_min=sol.root
177     print(d_min)

```

Listing 3: 螺距搜索程序

2.4 调头路径与速度约束程序

```

1 import numpy as np
2 import pandas as pd
3 import matplotlib.pyplot as plt
4 from scipy.interpolate import interp1d
5 from scipy.optimize import brentq
6
7
8

```

```

9 d=1.7
10 a=0
11 v0=1
12 R=4.5
13 num_nodes=224
14 # 把手距离
15 hand_length = 1.65 * np.ones(num_nodes - 1)
16 hand_length[0] = 2.86
17
18 theta0 =2 * np.pi * R / d
19
20 # 前距与边距
21 p = 0.275
22 q = 0.15
23
24 def get_vec_p(theta):
25     r = a + d / (2 * np.pi) * theta
26     dy=d/(2*np.pi)*np.sin(theta)+r*np.cos(theta)
27     dx=d/(2*np.pi)*np.cos(theta)-r*np.sin(theta)
28     sum2=np.sqrt(dx**2+dy**2)
29     p0=dx/sum2
30     p1=dy/sum2
31     return p0,p1
32
33 def get_alpha():
34     vec0=np.cos(theta0)
35     vec1=np.sin(theta0)
36     p0,p1=get_vec_p(theta0)
37     alpha=np.arccos(p0*vec0+p1*vec1)
38     return np.pi/2-alpha
39
40 alpha = get_alpha()
41
42 def equation(theta,t):
43     def f(theta):
44         return d/(2*np.pi)*(0.5*theta*np.sqrt(theta**2+1)+0.5*np.log(theta+np.sqrt(theta**2+1)))
45     return (f(theta)-f(theta0))/v0-t
46
47 def inverse(t):
48     def F(theta):
49         def f(theta):
50             return d/(2*np.pi)*(0.5*theta*np.sqrt(theta**2+1)+0.5*np.log(theta+np.sqrt(theta**2+1)))
51         return (f(theta) - f(theta0)) / v0
52
53
54     theta1=32*np.pi
55     theta_span=np.linspace(theta0,theta1,1000)
56     t_span=F(theta_span)
57     approx=interp1d(t_span,theta_span)
58
59     return approx(t)
60
61 def get_theta_head(t):
62     t=-t
63     theta0 = inverse(t)
64     theta1= brentq(lambda theta: equation(theta, t), theta0-np.pi/2, theta0+np.pi/2)

```

```

65     return theta1
66
67
68 def get_q_loc():
69     p0, p1 = get_vec_p(theta0)
70     r1 = 2 / 3 * R / np.cos(alpha)
71     r2 = 1 / 3 * R / np.cos(alpha)
72     # 点的坐标
73     x_p0 = d / (2 * np.pi) * theta0 * np.cos(theta0)
74     y_p0 = d / (2 * np.pi) * theta0 * np.sin(theta0)
75     x_p1 = -x_p0
76     y_p1 = -y_p0
77     x_q0 = x_p0 - r1 * p1
78     y_q0 = y_p0 - r1 * (-p0)
79     x_q1 = x_p1 + r2 * p1
80     y_q1 = y_p1 - r2 * p0
81
82     return x_q0, y_q0, x_q1, y_q1
83
84 def draw():
85     n=16
86     theta = np.linspace(theta0, 2 * n * np.pi, 1000)
87
88     r = d/(2*np.pi) * theta
89     theta1=theta+np.pi
90
91     x = r * np.cos(theta)
92     y = r * np.sin(theta)
93     x_R=R * np.cos(theta)
94     y_R=R * np.sin(theta)
95     x1=r * np.cos(theta1)
96     y1=r * np.sin(theta1)
97
98     x_line=np.linspace(R*np.cos(theta0),-R*np.cos(theta0), 1000)
99     y_line=np.tan(theta0)*x_line
100
101     plt.plot(x,y,x1, y1,lw=0.5)
102     plt.plot(x_R, y_R,lw=0.5)
103     plt.plot(x_line, y_line,lw=0.5)
104
105     p0, p1 = get_vec_p(theta0)
106     r0 = 2 / 3 * R / np.cos(alpha)
107     r1 = 1 / 3 * R / np.cos(alpha)
108     x_q0, y_q0, x_q1, y_q1=get_q_loc()
109     xs = [x_q0+r0*p1, x_q0, x_q1, x_q1-r1*p1]
110     ys = [y_q0+r0*(-p0), y_q0, y_q1,y_q1+r1*p0]
111
112
113     theta1=np.arctan((y_q1-y_q0)/(x_q1-x_q0))
114     rad1_span=np.linspace(theta1,theta1+np.pi-2*alpha,1000)
115     rad2_span=np.linspace(theta1+np.pi,theta1+2*np.pi-2*alpha,1000)
116     x_rad1=r0*np.cos(rad1_span)+x_q0
117     y_rad1=r0*np.sin(rad1_span)+y_q0
118     x_rad2=r1*np.cos(rad2_span)+x_q1
119     y_rad2=r1*np.sin(rad2_span)+y_q1
120     plt.plot(x_rad2,y_rad2,lw=0.5)

```

```

121 plt.plot(x_rad1,y_rad1,lw=0.5)
122
123
124 # 连线
125 plt.plot(xs, ys, color="blue",lw=0.5)
126
127 plt.axis('equal')
128
129
130
131 def get_xy(t):
132
133     t0 = 2 / 3 * (np.pi-2*alpha) * R / v0 / np.cos(alpha)
134     t1 = (np.pi-2*alpha) * R / v0 / np.cos(alpha)
135     r0 = 2 / 3 * R / np.cos(alpha)
136     r1 = 1 / 3 * R / np.cos(alpha)
137     x_q0, y_q0, x_q1, y_q1=get_q_loc()
138     theta1=np.arctan((y_q1-y_q0)/(x_q1-x_q0))
139
140
141     if t<=0:
142         theta=get_theta_head(t)
143         r = d / (2 * np.pi) * theta
144         x = r * np.cos(theta)
145         y = r * np.sin(theta)
146     elif 0<t<=t0:
147         dtheta=v0*t/r0
148         theta=theta1+np.pi-2*alpha-dtheta
149         x = r0 * np.cos(theta) + x_q0
150         y = r0 * np.sin(theta) + y_q0
151     elif t0<t<t1:
152         dtheta = v0 * (t-t0)/ r1
153         theta = theta1+np.pi+dtheta
154         x = r1 * np.cos(theta) + x_q1
155         y = r1 * np.sin(theta) + y_q1
156     else:
157         theta = get_theta_head(t1 - t)
158         r = d / (2 * np.pi) * theta
159         x = -r * np.cos(theta)
160         y = -r * np.sin(theta)
161     return x,y
162
163 def get_vec_p_abt_t(t):
164     t0 = 2 / 3 * (np.pi-2*alpha) * R / v0 / np.cos(alpha)
165     t1 = (np.pi-2*alpha) * R / v0 / np.cos(alpha)
166     r0 = 2 / 3 * R / np.cos(alpha)
167     r1 = 1 / 3 * R / np.cos(alpha)
168     x_q0, y_q0, x_q1, y_q1 = get_q_loc()
169     theta1 = np.arctan((y_q1 - y_q0) / (x_q1 - x_q0))
170     if t<=0:
171         theta=get_theta_head(t)
172         dx=np.cos(theta)-theta*np.sin(theta)
173         dy=np.sin(theta)+theta*np.cos(theta)
174         sum=np.sqrt(dx**2+dy**2)
175         p0=-dx/sum
176         p1=-dy/sum

```

```

177 elif 0<t<=t0:
178     dtheta = v0 * t / r0
179     theta = theta1 + np.pi - 2 * alpha - dtheta
180     p0=np.sin(theta)
181     p1=-np.cos(theta)
182 elif t0<t<t1:
183     dtheta = v0 * (t - t0) / r1
184     theta = theta1 + np.pi + dtheta
185     p0 = -np.sin(theta)
186     p1 = np.cos(theta)
187 else:
188     theta = get_theta_head(t1-t)
189     dx = np.cos(theta) - theta * np.sin(theta)
190     dy = np.sin(theta) + theta * np.cos(theta)
191     sum = np.sqrt(dx ** 2 + dy ** 2)
192     p0 = -dx / sum
193     p1 = -dy / sum
194 return p0,p1
195
196 def draw_body(x,y):
197     """
198     法向量和切向量
199     """
200
201     dx=x[:-1] - x[1:]
202     dy=y[:-1] - y[1:]
203     lens=np.sqrt(dx**2+dy ** 2)
204     vec_n = np.vstack((dx/lens, dy/lens))
205
206     vec_t = np.vstack((-vec_n[1, :], vec_n[0, :]))
207
208     """
209     四个角点坐标
210     """
211     points_out1 = np.vstack((x[:-1], y[:-1])) + p * vec_n + q * vec_t
212     points_out2 = np.vstack((x[1:], y[1:])) - p * vec_n + q * vec_t
213     points_in1 = np.vstack((x[:-1], y[:-1])) + p * vec_n - q * vec_t
214     points_in2 = np.vstack((x[1:], y[1:])) - p * vec_n - q * vec_t
215
216     for i in range(num_nodes - 1):
217         # 四边形顶点按顺序连接
218         x_poly = [points_out1[0, i], points_out2[0, i], points_in2[0, i], points_in1[0, i],
219                 points_out1[0, i]]
220         y_poly = [points_out1[1, i], points_out2[1, i], points_in2[1, i], points_in1[1, i],
221                 points_out1[1, i]]
222
223         plt.plot(x_poly, y_poly, 'k-', lw=0.5)
224         # 填充棕色内部
225         plt.fill(x_poly, y_poly, color='brown', alpha=0.5)
226
227 if __name__ == '__main__':
228     data_xy=np.zeros((2*num_nodes,201))
229     data_v=np.zeros((num_nodes,201))
230     data_v[0,:]=v0
231     t_span=np.arange(-100,101,1)

```

```

231
232
233 for j,t_pre in enumerate(t_span):
234     x_pre, y_pre = get_xy(t_pre)
235     data_xy[0, j] = x_pre
236     data_xy[1, j] = y_pre
237     vec_p0_pre, vec_p1_pre = get_vec_p_abt_t(t_pre)
238     def f(t):
239         return np.sqrt((get_xy(t)[0] - x_pre) ** 2 + (get_xy(t)[1] - y_pre) ** 2) - hand_length
240
241
242     t_back = brentq(f,t_pre - 6,t_pre,xtol=1e-10)
243
244     x_back, y_back = get_xy(t_back)
245     data_xy[2, j] = x_back
246     data_xy[3, j] = y_back
247     vec_p0_back, vec_p1_back = get_vec_p_abt_t(t_back)
248
249     vec_n0=x_pre-x_back
250     vec_n1=y_pre-y_back
251     sum=np.sqrt(vec_n0**2+vec_n1**2)
252     vec_n0=vec_n0/sum
253     vec_n1=vec_n1/sum
254
255     rate=abs((vec_p0_pre*vec_n0+vec_p1_pre*vec_n1)/(vec_p0_back*vec_n0+vec_p1_back*vec_n1))
256     data_v[1, j]=data_v[0, j]*rate
257
258     x_pre = x_back
259     y_pre = y_back
260     t_pre = t_back
261     vec_p1_pre=vec_p1_back
262     vec_p0_pre=vec_p0_back
263
264
265     def g(t, x_pre, y_pre):
266         return np.sqrt((get_xy(t)[0] - x_pre) ** 2 + (get_xy(t)[1] - y_pre) ** 2) - hand_length
267
268
269     for i in range(2, num_nodes):
270         t_back = brentq(lambda t: g(t, x_pre, y_pre), t_pre - 3, t_pre, xtol=1e-10)
271         x_back, y_back = get_xy(t_back)
272         data_xy[2 * i, j] = x_back
273         data_xy[2 * i + 1, j] = y_back
274         vec_p0_back, vec_p1_back = get_vec_p_abt_t(t_back)
275
276         vec_n0 = x_pre - x_back
277         vec_n1 = y_pre - y_back
278         sum = np.sqrt(vec_n0 ** 2 + vec_n1 ** 2)
279         vec_n0 = vec_n0 / sum
280         vec_n1 = vec_n1 / sum
281
282         data_v[i, j] = data_v[i-1, j] * abs((vec_p0_pre * vec_n0 + vec_p1_pre * vec_n1) / (
283             vec_p0_back * vec_n0 + vec_p1_back * vec_n1))

```

```

284         x_pre = x_back
285         y_pre = y_back
286         t_pre = t_back
287         vec_p1_pre = vec_p1_back
288         vec_p0_pre = vec_p0_back
289
290     plt.plot(data_xy[0:2 * num_nodes:2, 131], data_xy[1:2 * num_nodes:2, 131], lw=0.5)
291     draw()
292     draw_body(data_xy[0:2 * num_nodes:2, 131], data_xy[1:2 * num_nodes:2, 131])
293
294     row_names_xy=[]
295     row_names_v=[]
296     for i in range(num_nodes):
297         row_names_xy.append(f'Node_{i}_x')
298         row_names_xy.append(f'Node_{i}_y')
299         row_names_v.append(f'Node_{i}_v')
300
301     col_names=list(range(-100,101,1))
302
303     df_xy=pd.DataFrame(data_xy,index=row_names_xy,columns=col_names)
304     df_v=pd.DataFrame(data_v,index=row_names_v,columns=col_names)
305     df_xy.to_excel("result4_xy.xlsx")
306     df_v.to_excel("result4_v.xlsx")
307
308     plt.show()

```

Listing 4: 调头路径与速度约束程序

2.5 补充参考程序

```

1 from pandas.core.interchange.dataframe_protocol import DataFrame
2 import calcul
3 import pandas as pd
4
5
6 times=[0,60,120,180,240,300]
7 head_thetas=[]
8 for t in times:
9     head=calcul.find_head(t)
10    head_thetas.append(head)
11
12 head_xys=[(880,0)]
13 for theta in head_thetas[1:]:
14     x,y=calcul.to_xy(theta)
15     head_xys.append((x,y))
16
17 list1=[]
18 for i in range(6):
19     theta_h=head_thetas[i]
20     ans=[]
21     ans.append(theta_h)
22     last=calcul.find_next(theta_h,286.0)
23     for j in range(223):
24         if last is None:

```

```

25         ans.append(None)
26     else:
27         ans.append(last)
28         last=calcu.find_next(last,165.0)
29     list1.append(ans)
30
31 ans_list1=[]
32 myindex=[0,1,51,101,151,201,223]
33 for i in range(6):
34     ans=[]
35     for j in myindex:
36         x,y=calcu.to_xy(list1[i][j])
37         ans.append(x/100)
38         ans.append(y/100)
39     ans_list1.append(ans)
40
41 #print(ans_list1)
42 '''
43 df1=pd.DataFrame(ans_list1).T
44 df1.columns=['0s','60s','120s','180s','240s','300s']
45 df1.index=[
46     '龙头 x (m)',
47     '龙头 y (m)',
48     '第 1 节龙身 x (m)',
49     '第 1 节龙身 y (m)',
50     '第 51 节龙身 x (m)',
51     '第 51 节龙身 y (m)',
52     '第 101 节龙身 x (m)',
53     '第 101 节龙身 y (m)',
54     '第 151 节龙身 x (m)',
55     '第 151 节龙身 y (m)',
56     '第 201 节龙身 x (m)',
57     '第 201 节龙身 y (m)',
58     '龙尾 (后) x (m)',
59     '龙尾 (后) y (m)'
60 ]
61 with pd.ExcelWriter('result1.xlsx') as result1:
62     df1.to_excel(result1)
63 '''
64
65 list2=[]
66 for i in range(6):
67     ans=[]
68     v_A=1.0
69     ans.append(v_A)
70     for j in range(223):
71         theta_A=list1[i][j]
72         theta_B=list1[i][j+1]
73         v_B=calcu.next_velocity(theta_A,theta_B,v_A)
74         ans.append(v_B)
75         v_A=v_B
76     list2.append(ans)
77
78 ans_list2=[]
79 for i in range(6):
80     ans=[]

```

```

81     for j in myindex:
82         ans.append(list2[i][j])
83     ans_list2.append(ans)
84
85 '''
86 df2=pd.DataFrame(ans_list2).T
87 df2.columns=['0s','60s','120s','180s','240s','300s']
88 df2.index=[
89     '龙头',
90     '第 1 节龙身',
91     '第 51 节龙身',
92     '第 101 节龙身',
93     '第 151 节龙身',
94     '第 201 节龙身',
95     '龙尾（后）',
96 ]
97 with pd.ExcelWriter('result1_2.xlsx') as result2:
98     df2.to_excel(result2)
99 '''

```

Listing 5: 补充参考程序